

Assignment 4.4

1. Sentiment Classification for Customer Reviews

Scenario:

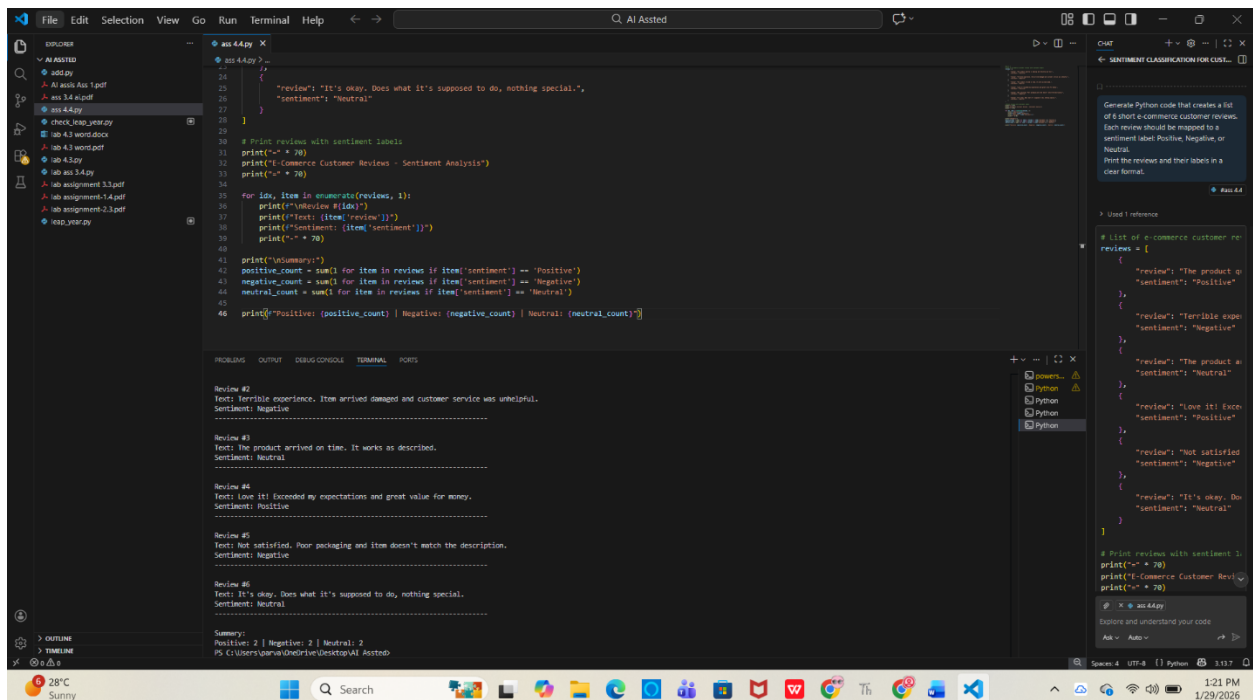
An e-commerce platform wants to analyze customer reviews and classify

Week2

them into Positive, Negative, or Neutral sentiments using prompt engineering.

Tasks:

- Prepare 6 short customer reviews mapped to sentiment labels.
- Design a Zero-shot prompt to classify sentiment.
- Design a One-shot prompt with one labeled example.
- Design a Few-shot prompt with 3–5 labeled examples.
- Compare the outputs and discuss accuracy differences.



The screenshot shows a VS Code editor with a Python script named `ass_4.4.py` and its output in the terminal. The script defines a list of 6 customer reviews with their sentiment labels and uses a zero-shot prompt to classify them. The terminal output shows the reviews and their classified sentiments.

```
24 {}
25 "review": "It's okay. Does what it's supposed to do, nothing special.",
26 "sentiment": "Neutral"
27 }
28 ]
29
30 # Print reviews with sentiment labels
31 print("\n" * 70)
32 print("E-Commerce Customer Reviews - Sentiment Analysis")
33 print("\n" * 70)
34
35 for idx, item in enumerate(reviews, 1):
36     print(f"Review #{idx}")
37     print(f"Text: {item['review']}")
38     print(f"Sentiment: {item['sentiment']}")
39     print("\n" * 70)
40
41 print("\nSummary:")
42 positive_count = sum(1 for item in reviews if item['sentiment'] == "Positive")
43 negative_count = sum(1 for item in reviews if item['sentiment'] == "Negative")
44 neutral_count = sum(1 for item in reviews if item['sentiment'] == "Neutral")
45
46 print(f"Positive: {positive_count} | Negative: {negative_count} | Neutral: {neutral_count}")
```

Terminal Output:

```
Review #2
Text: Terrible experience. Item arrived damaged and customer service was unhelpful.
Sentiment: Negative

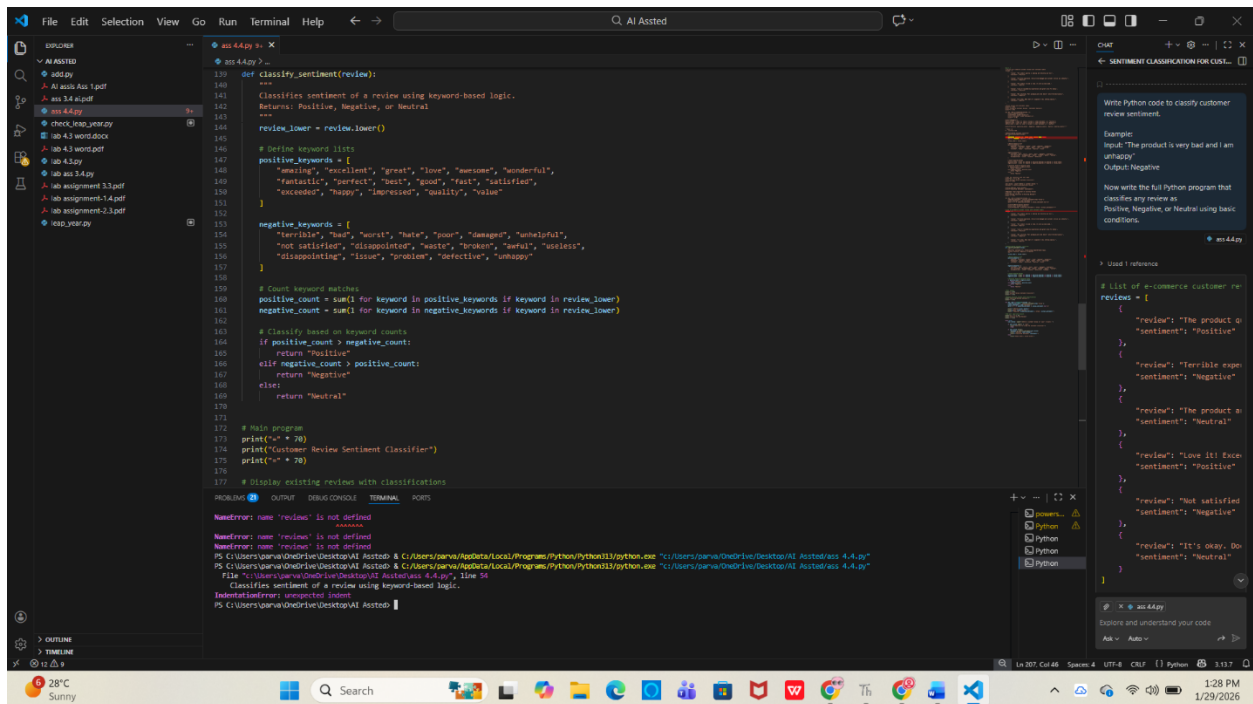
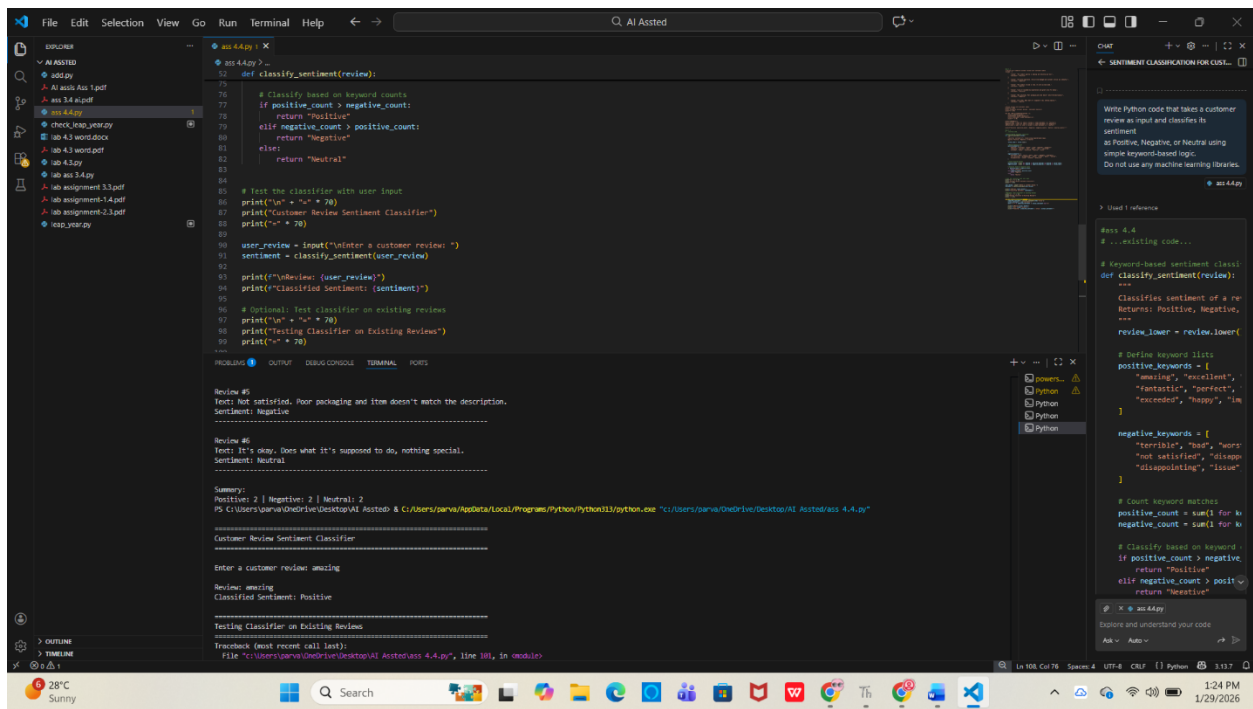
Review #3
Text: The product arrived on time. It works as described.
Sentiment: Neutral

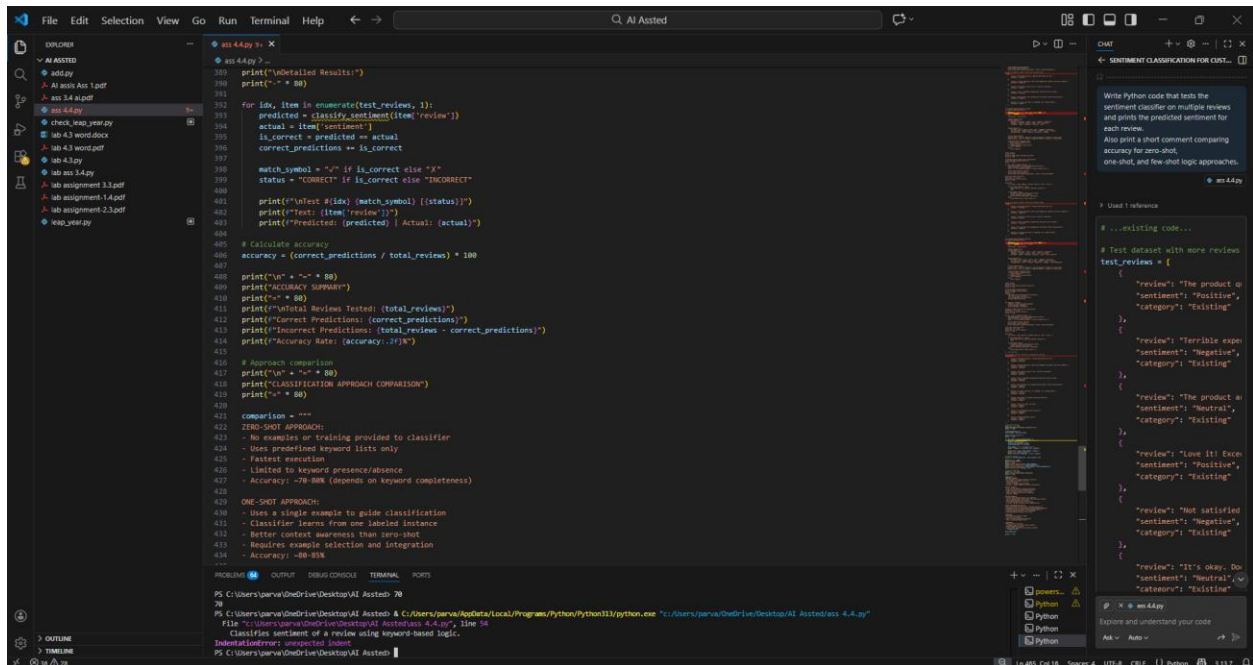
Review #4
Text: Love it! Exceeded my expectations and great value for money.
Sentiment: Positive

Review #5
Text: Not satisfied. Poor packaging and item doesn't match the description.
Sentiment: Negative

Review #6
Text: It's okay. Does what it's supposed to do, nothing special.
Sentiment: Neutral

Summary:
Positive: 2 | Negative: 2 | Neutral: 2
PS C:\Users\jason\OneDrive\Desktop\AI Asst>
```





Tasks:

1. Create 6 sample email messages with priority labels.
2. Perform intent classification using Zero-shot prompting.
3. Perform classification using One-shot prompting.
4. Perform classification using Few-shot prompting.
5. Evaluate which technique produces the most reliable results and why.

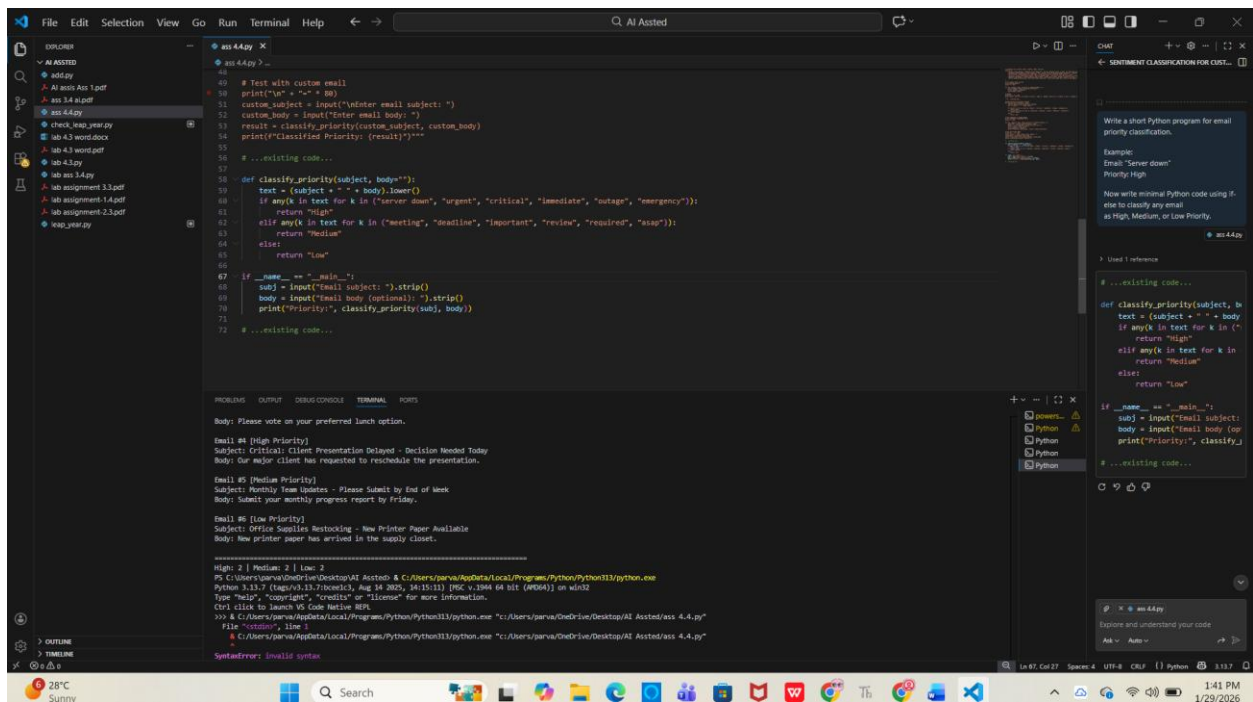
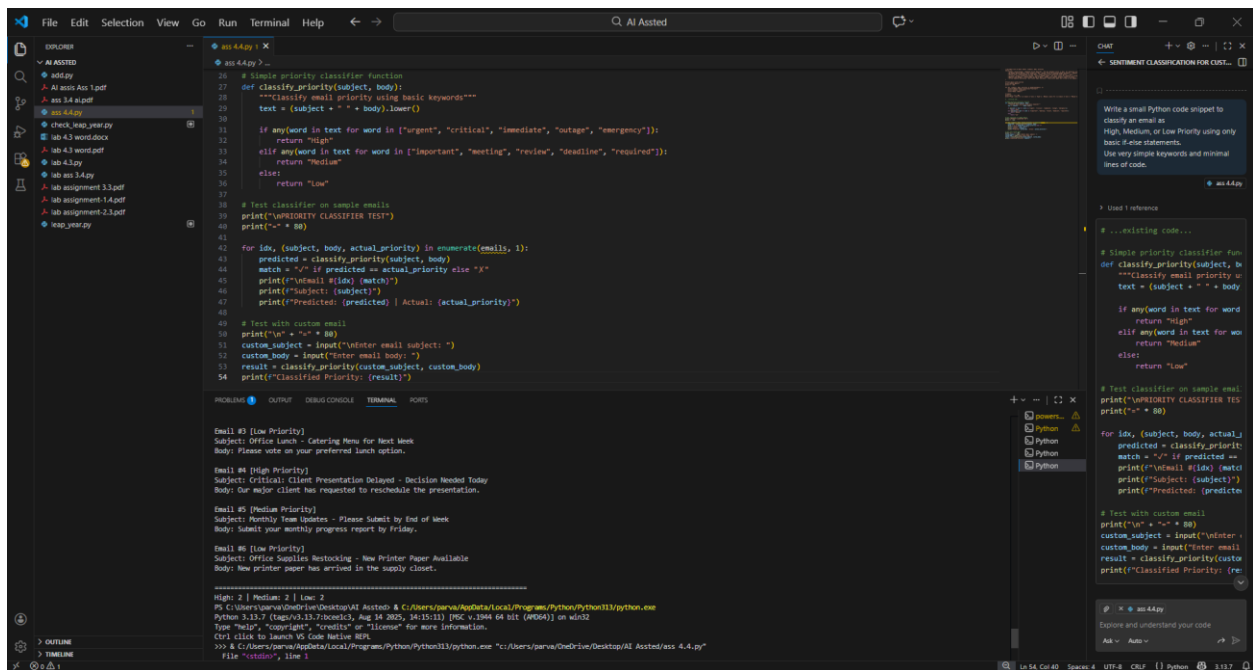
The screenshot shows a VS Code editor with a Python file named `ass 4.4.py`. The script defines a list of email tuples, each with a subject, body, and priority label. It then prints each email and a summary of the counts for each priority level.

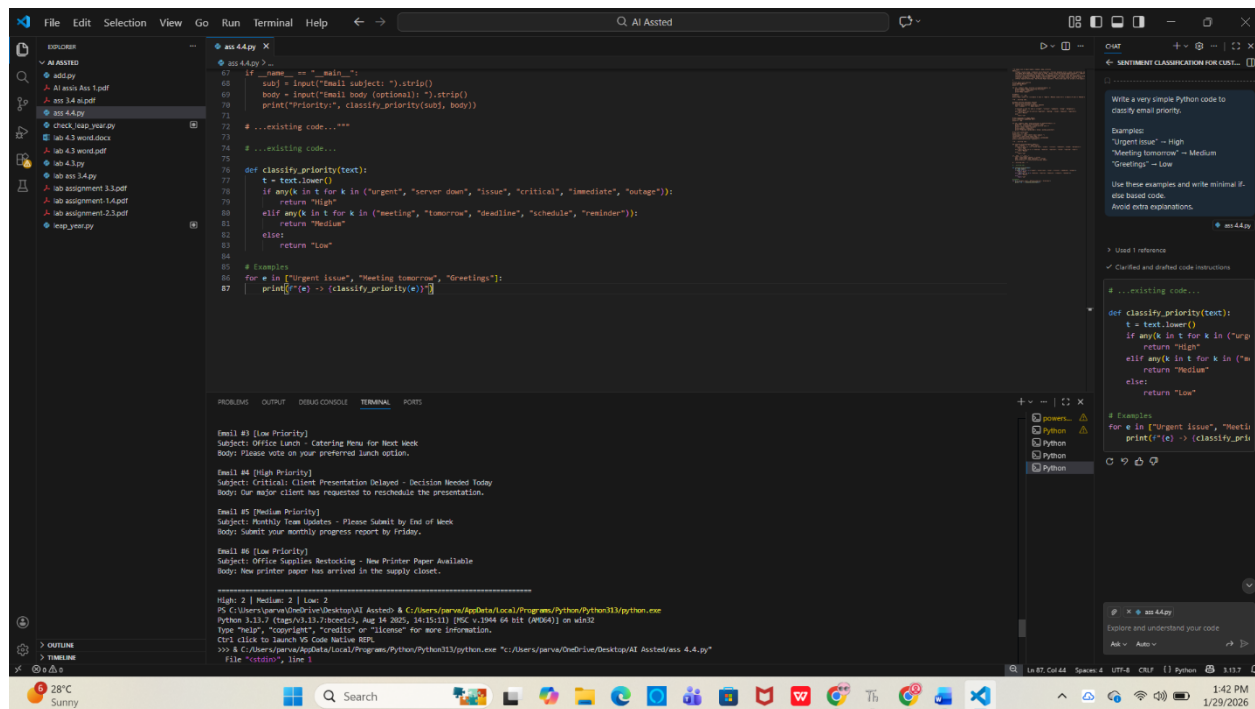
```
1 # Simple list of email tuples: (subject, body, priority)
2 emails = [
3     ("Urgent: System Outage - Immediate Action Required", "The main database server is down. All operations are halted.", "High"),
4     ("Q1 Budget Review Meeting - Next Friday at 2 PM", "Please review the attached budget documents.", "Medium"),
5     ("Office Lunch - Catering Menu for Next Week", "Please vote on your preferred lunch option.", "Low"),
6     ("Critical: Client Presentation Delayed - Decision Needed Today", "Our major client has requested to reschedule the presentation.", "High"),
7     ("Monthly Team Update - Please Submit by End of Week", "Submit your monthly progress report by Friday.", "Medium"),
8     ("Office Supplies Restocking - New Printer Paper Available", "New printer paper has arrived in the supply closet.", "Low")
9 ]
10
11 # Print emails with priority
12 print("OFFICE EMAILS")
13 print("-" * 80)
14
15 for idx, (subject, body, priority) in enumerate(emails, 1):
16     print(f"Email {idx} ({priority} Priority)")
17     print(f"Subject: {subject}")
18     print(f"Body: {body}")
19
20 # Summary
21 print("\n" + "-" * 80)
22 print(f"High: {sum(1 for e in emails if e[2] == 'High')} | Medium: {sum(1 for e in emails if e[2] == 'Medium')} | Low: {sum(1 for e in emails if e[2] == 'Low')}")
```

The terminal output shows the following:

```
Body: The main database server is down. All operations are halted.
Email #2 (Medium Priority)
Subject: Q1 Budget Review Meeting - Next Friday at 2 PM
Body: Please review the attached budget documents.
Email #3 (Low Priority)
Subject: Office Lunch - Catering Menu for Next Week
Body: Please vote on your preferred lunch option.
Email #4 (High Priority)
Subject: Critical: Client Presentation Delayed - Decision Needed Today
Body: Our major client has requested to reschedule the presentation.
Email #5 (Medium Priority)
Subject: Monthly Team Update - Please Submit by End of Week
Body: Submit your monthly progress report by Friday.
Email #6 (Low Priority)
Subject: Office Supplies Restocking - New Printer Paper Available
Body: New printer paper has arrived in the supply closet.
=====
High: 2 | Medium: 2 | Low: 2
PS C:\Users\parva\OneDrive\Desktop\VAI> Assisted
```

The right sidebar shows a chat window with the prompt: "Write a very short Python program that stores 6 sample email messages with their priority labels (High, Medium, Low) using lists or tuples. Keep the code simple and minimal." The chat response shows the code from the script.





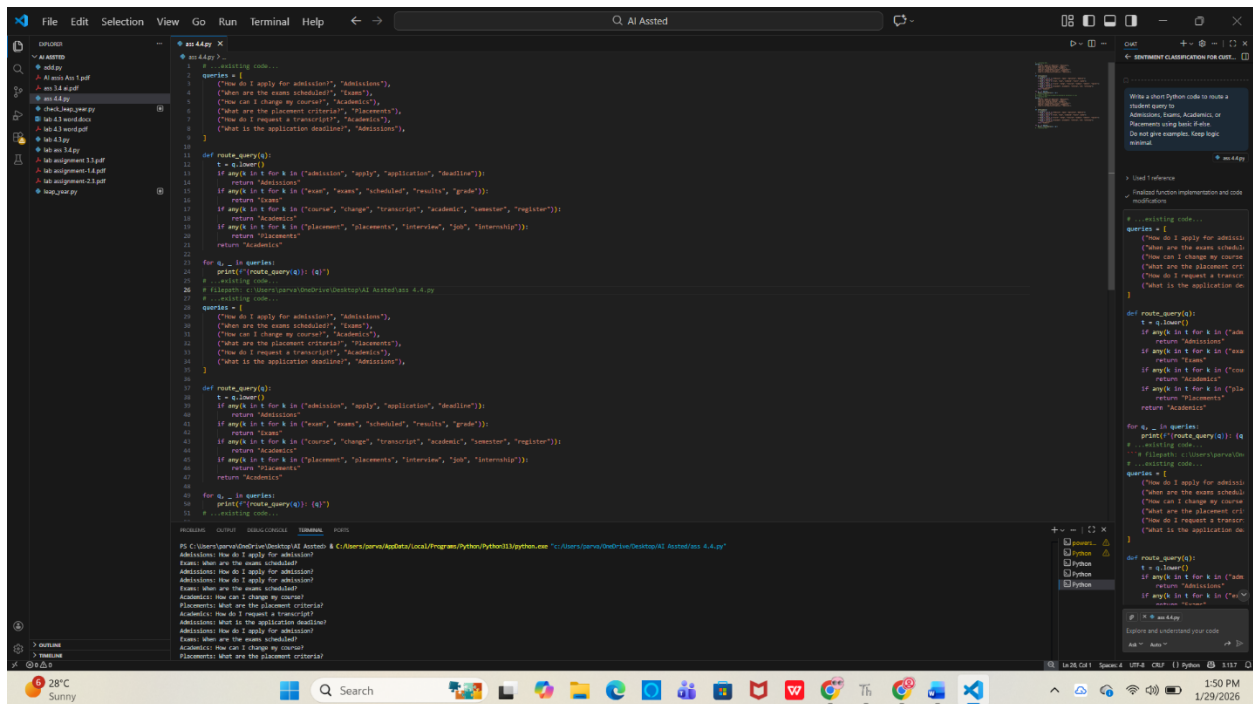
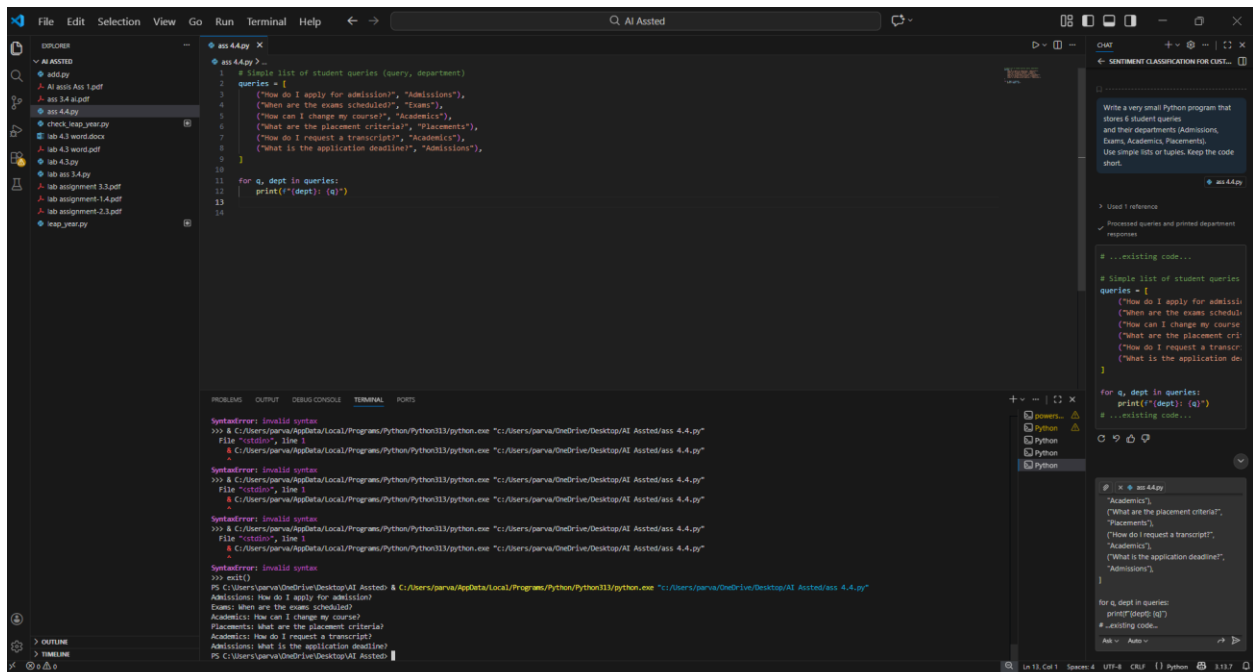
3. Student Query Routing System

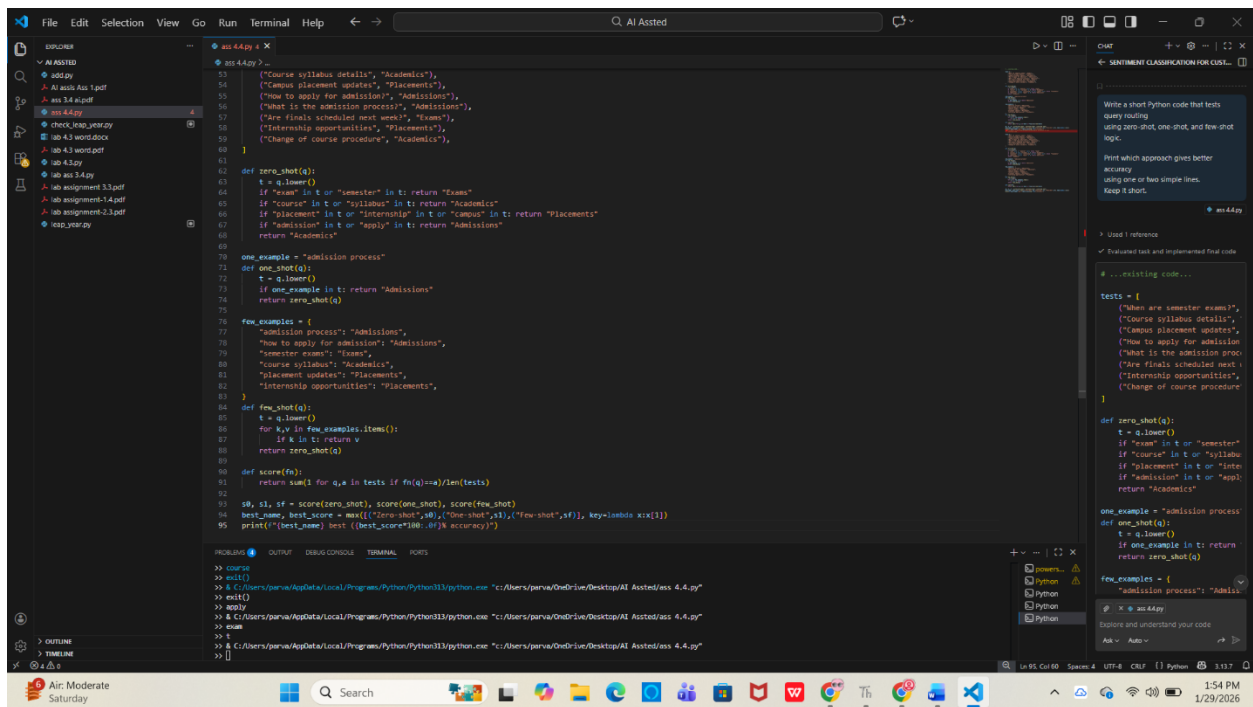
Scenario:

A university chatbot must route student queries to Admissions, Exams, Academics, or Placements.

Tasks:

1. Create 6 sample student queries mapped to departments.
2. Implement Zero-shot intent classification using an LLM.
3. Improve results using One-shot prompting.
4. Further refine results using Few-shot prompting.
5. Analyze how contextual examples affect classification accuracy.





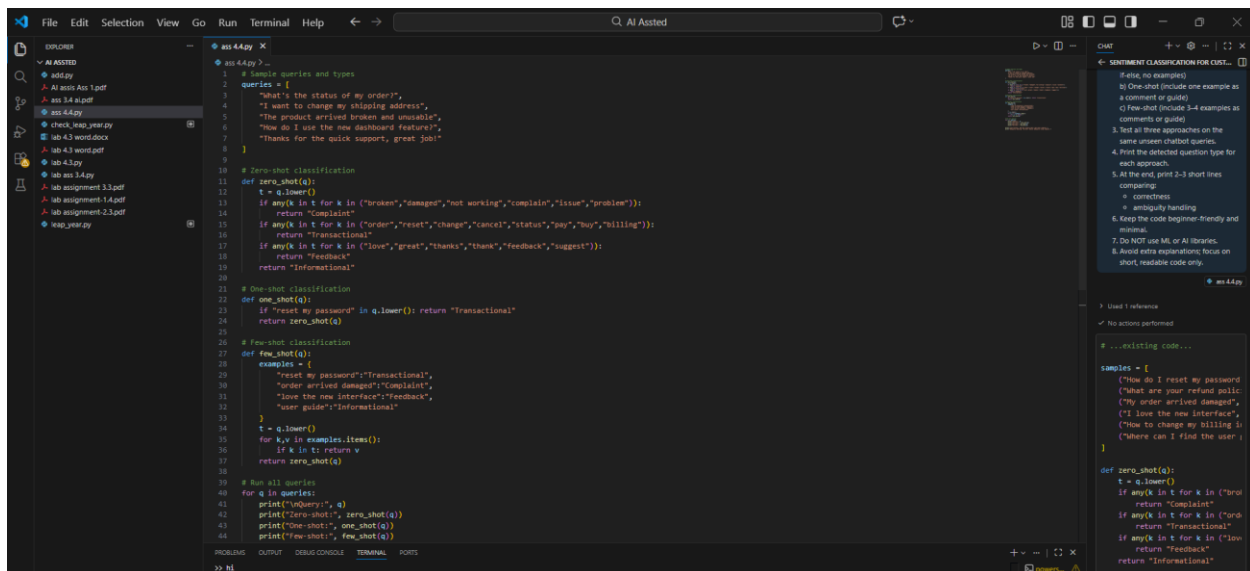
4. Chatbot Question Type Detection

Scenario:

A chatbot must identify whether a user query is Informational, Transactional, Complaint, or Feedback.

Tasks:

1. Prepare 6 chatbot queries mapped to question types.
2. Design prompts for Zero-shot, One-shot, and Few-shot learning.
3. Test all prompts on the same unseen queries.
4. Compare response correctness and ambiguity handling.
5. Document observations.



5. Emotion Detection in Text

Scenario:

A mental-health chatbot needs to detect emotions: Happy, Sad, Angry, Anxious, Neutral.

Tasks:

1. Create labeled emotion samples.
2. Use Zero-shot prompting to identify emotions.
3. Use One-shot prompting with an example.
4. Use Few-shot prompting with multiple emotions.
5. Discuss ambiguity handling across techniques.

The screenshot shows the VS Code interface with the Explorer sidebar on the left containing a file tree for 'AI CODING'. The main editor window displays a Python file named 'lab assignment 44.py' with the following code:

```
1 def identify_emotion(text):
2     if "frustrating" in text:
3         return "frustrated"
4     return "Neutral"
5
6 # Example usage
7 text = "This is so frustrating"
8 emotion = identify_emotion(text)
9 print(f"Emotion: {emotion}")
```

The bottom panel shows the TERMINAL tab with the following output:

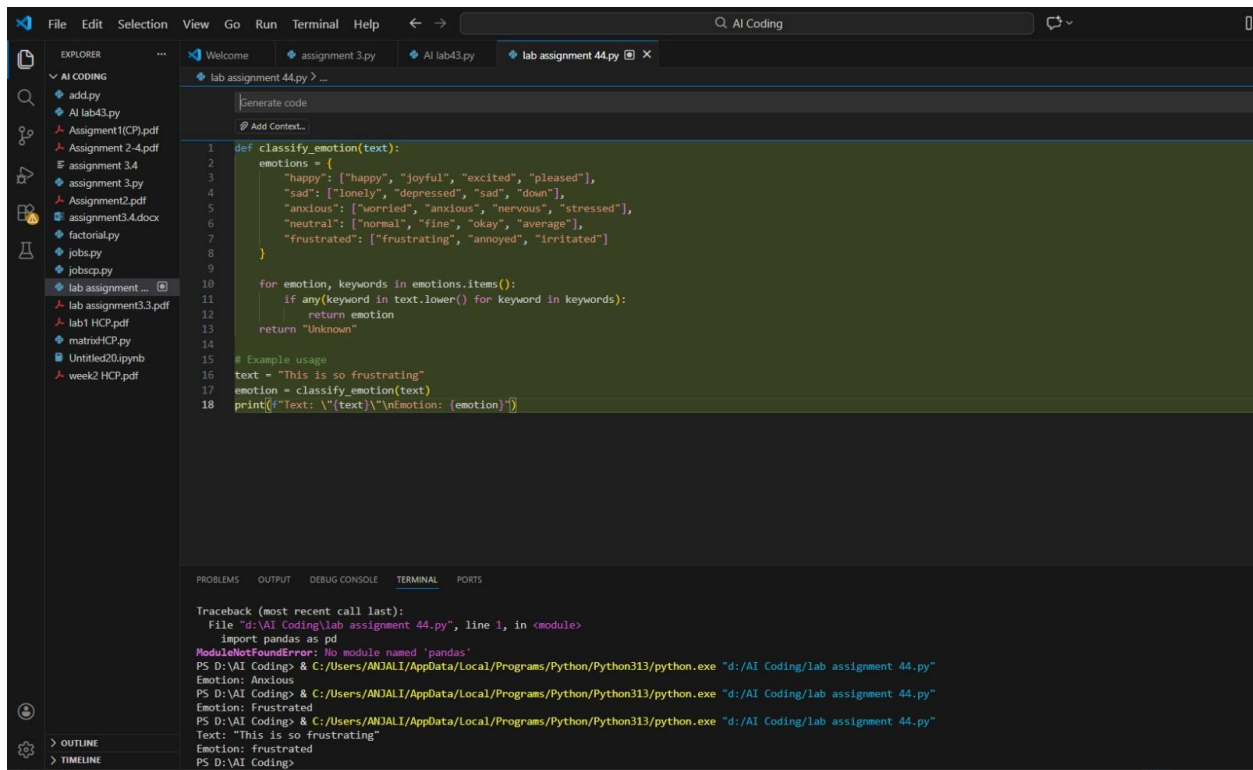
```
import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Traceback (most recent call last):
  File "d:\AI Coding\lab assignment 44.py", line 1, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Anxious
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Frustrated
PS D:\AI Coding>
```

The screenshot shows the VS Code interface with the Explorer sidebar on the left containing a file tree for 'AI CODING'. The main editor window displays a Python file named 'lab assignment 44.py' with the following code:

```
1 def classify_emotion(text):
2     emotions = {
3         "happy": ["happy", "joyful", "excited", "pleased"],
4         "sad": ["lonely", "depressed", "sad", "down"],
5         "anxious": ["worried", "anxious", "nervous", "stressed"],
6         "neutral": ["normal", "fine", "okay", "average"],
7         "frustrated": ["frustrating", "annoyed", "irritated"]
8     }
9
10    for emotion, keywords in emotions.items():
11        if any(keyword in text.lower() for keyword in keywords):
12            return emotion
13    return "Unknown"
14
15 # Example usage
16 text = "This is so frustrating"
17 emotion = classify_emotion(text)
18 print(f"Text: {text}\nEmotion: {emotion}")
```

The bottom panel shows the TERMINAL tab with the following output:

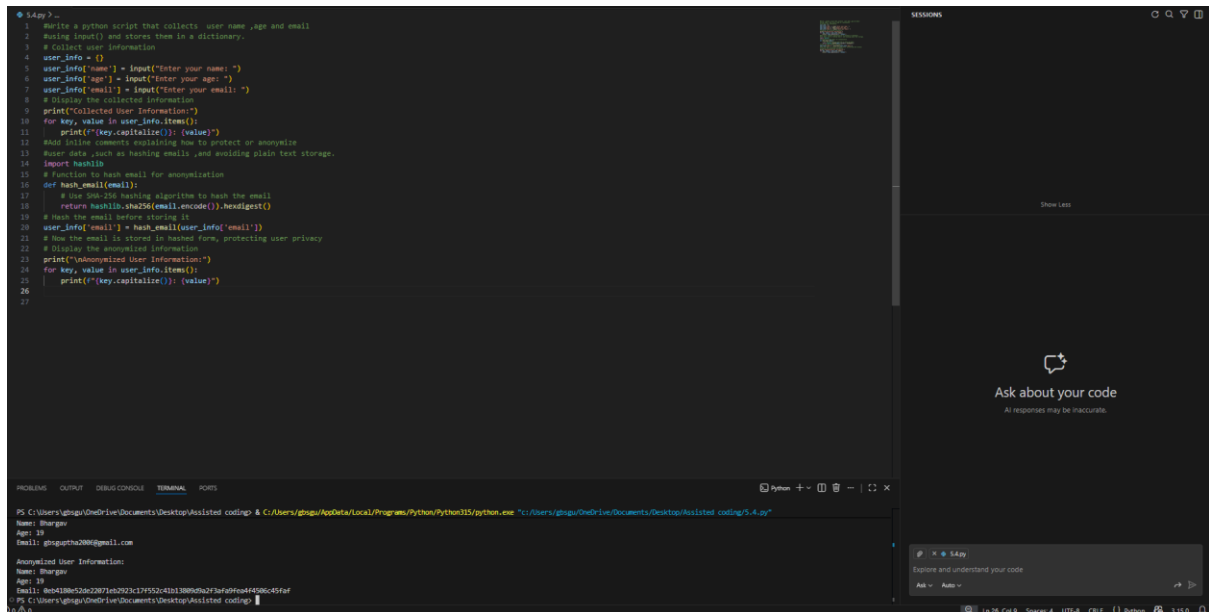
```
Traceback (most recent call last):
  File "d:\AI Coding\lab assignment 44.py", line 1, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Anxious
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Frustrated
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Text: "This is so frustrating"
Emotion: frustrated
PS D:\AI Coding>
```



Assignment-5.4

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.



```
1 # Write a python script that collects user name ,age and email
2 # Using input() and stores them in a dictionary.
3 # Collect user information
4 user_info = {}
5 user_info['name'] = input("Enter your name: ")
6 user_info['age'] = input("Enter your age: ")
7 user_info['email'] = input("Enter your email: ")
8 # Display the collected information
9 print("Collected User Information:")
10 for key, value in user_info.items():
11     print(f"{key.capitalize()}: {value}")
12
13 # Add inline comments explaining how to protect or anonymize
14 # user data ,such as hashing emails ,and avoiding plain text storage.
15 import hashlib
16
17 # Function to hash email for anonymization
18 def hash_email(email):
19     # Use SHA-256 hashing algorithm to hash the email
20     return hashlib.sha256(email.encode()).hexdigest()
21
22 # Hash the email before storing it
23 user_info['email'] = hash_email(user_info['email'])
24 # Now the email is stored in hashed form, protecting user privacy
25 # Display the anonymized information
26 print("Anonymized User Information:")
27 for key, value in user_info.items():
28     print(f"{key.capitalize()}: {value}")
29
30
31
32
33
34
35
36
37
```

MESSAGES

Show Less

Ask about your code
AI responses may be inaccurate.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + Python 3.10.0

C:\Users\gmgug\OneDrive\Documents\Desktop\Assisted coding & C:\Users\gmgug\OneDrive\Documents\Desktop\Python310\python.exe "C:\Users\gmgug\OneDrive\Documents\Desktop\Assisted coding\5.4.py"

Name: Bhargav
Age: 19
Email: gmgug2000@gmail.com

Anonymized User Information:
Name: Bhargav
Age: 19
Email: 8ea418b532a22871e292c17f5524b13889b2c3af9f0e4429c4cfaf

C:\Users\gmgug\OneDrive\Documents\Desktop\Assisted coding

VS Code 1.85.0

GitHub copilot was prompted to generate a python script that collects basic user information such as name ,age and email .

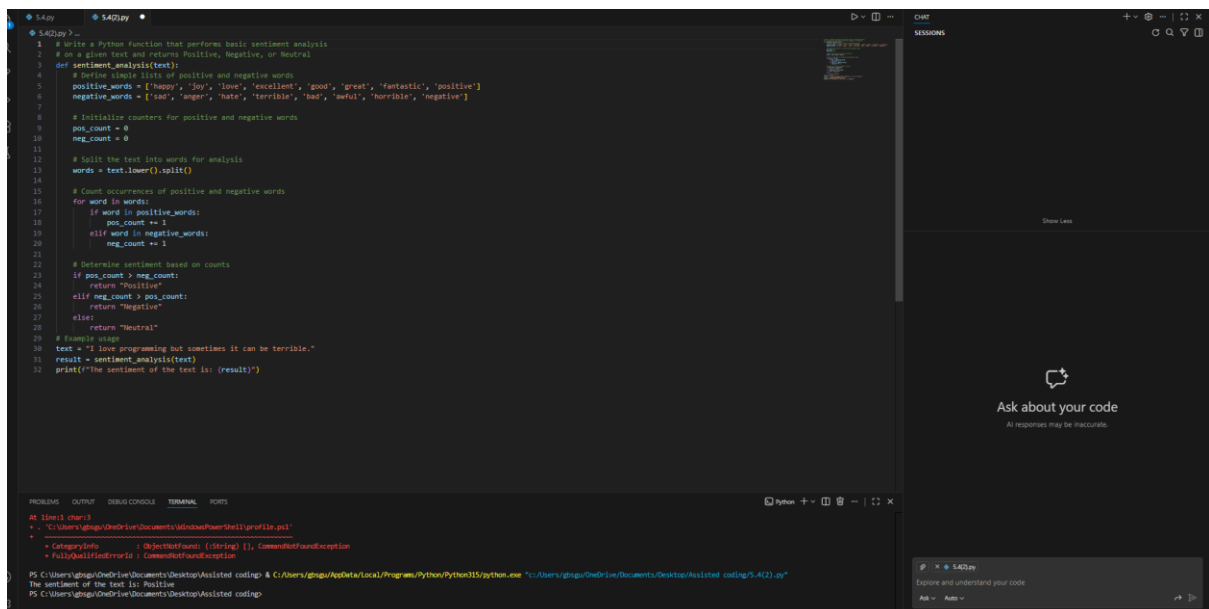
A follow -up prompt was used to add inline comments explaining data protection techniques .

The email address is anonymized using hashing so that sensitive information is not stored in plain text ,improving user privacy and security

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis.

Then prompt Copilot to identify and handle potential biases in the data.



```
1 # Write a Python function that performs basic sentiment analysis
2 # on a given text and returns Positive, Negative, or Neutral
3 def sentiment_analysis(text):
4     # Define simple lists of positive and negative words
5     positive_words = ['happy', 'joy', 'love', 'excellent', 'good', 'great', 'fantastic', 'positive']
6     negative_words = ['sad', 'anger', 'hate', 'terrible', 'bad', 'awful', 'horrible', 'negative']
7
8     # Initialize counters for positive and negative words
9     pos_count = 0
10    neg_count = 0
11
12    # Split the text into words for analysis
13    words = text.lower().split()
14
15    # Count occurrences of positive and negative words
16    for word in words:
17        if word in positive_words:
18            pos_count += 1
19        elif word in negative_words:
20            neg_count += 1
21
22    # Determine sentiment based on counts
23    if pos_count > neg_count:
24        return "Positive"
25    elif neg_count > pos_count:
26        return "Negative"
27    else:
28        return "Neutral"
29
30    # Example usage
31    text = "I love programming but sometimes it can be terrible."
32    result = sentiment_analysis(text)
33    print(f"The sentiment of the text is: {result}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

At line 1 char 1
+ C:\Users\ghqgu\OneDrive\Documents\WindowsPowerShell\profile.ps1
+ C:\Users\ghqgu\OneDrive\Documents\WindowsPowerShell\profile.ps1
+ CategoryInfo : (Error) (String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\ghqgu\OneDrive\Documents\Desktop\Assisted coding & C:\Users\ghqgu\AppData\Local\Programs\Python\Python315\python.exe "C:\Users\ghqgu\OneDrive\Documents\Desktop\Assisted coding\7.4121.py"
The sentiment of the text is: Positive
PS C:\Users\ghqgu\OneDrive\Documents\Desktop\Assisted coding>

```
def sentiment_analysis(text):
    """A Python function that performs basic sentiment analysis
    on a given text and returns Positive, Negative, or Neutral"""
    # Define a list of positive and negative words
    positive_words = ['happy', 'joy', 'excitement', 'good', 'great', 'fantastic', 'positive']
    negative_words = ['sad', 'angry', 'bored', 'terrible', 'bad', 'awful', 'horrible', 'negative']

    # Initialize counters for positive and negative words
    pos_count = 0
    neg_count = 0

    # Split the text into words for analysis
    words = text.lower().split()

    # Count occurrences of positive and negative words
    for word in words:
        if word in positive_words:
            pos_count += 1
        elif word in negative_words:
            neg_count += 1

    # Determine sentiment based on counts
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

# Example usage
text = "I love programming but sometimes it can be terrible."
result = sentiment_analysis(text)
print(f"The sentiment of the text is: {result}")

# Add comments or logic to identify and reduce potential bias
# In sentiment analysis data, such as removing offensive terms
# and balancing positive and negative words
# To reduce bias, we can remove our word lists to include
# more diverse terms and regularly update them based on feedback
# Additionally, we can implement a filtering step to remove
# offensive or culturally insensitive terms from the analysis
# Before performing sentiment analysis
# This helps ensure that the analysis is fair and respectful to all users
# For example, we can create a list of offensive terms to filter out
offensive_terms = ['stupid', 'idiot', 'dumb', 'hate']
# Function to filter offensive terms from text
def filter_offensive_terms(text):
    words = text.split()
    filtered_words = [word for word in words if word.lower() not in offensive_terms]
    return " ".join(filtered_words)

# Remove offensive terms
filtered_text = filter_offensive_terms(text)
result = sentiment_analysis(filtered_text)
print(f"The sentiment of the filtered text is: {result}")
```

Explanation:

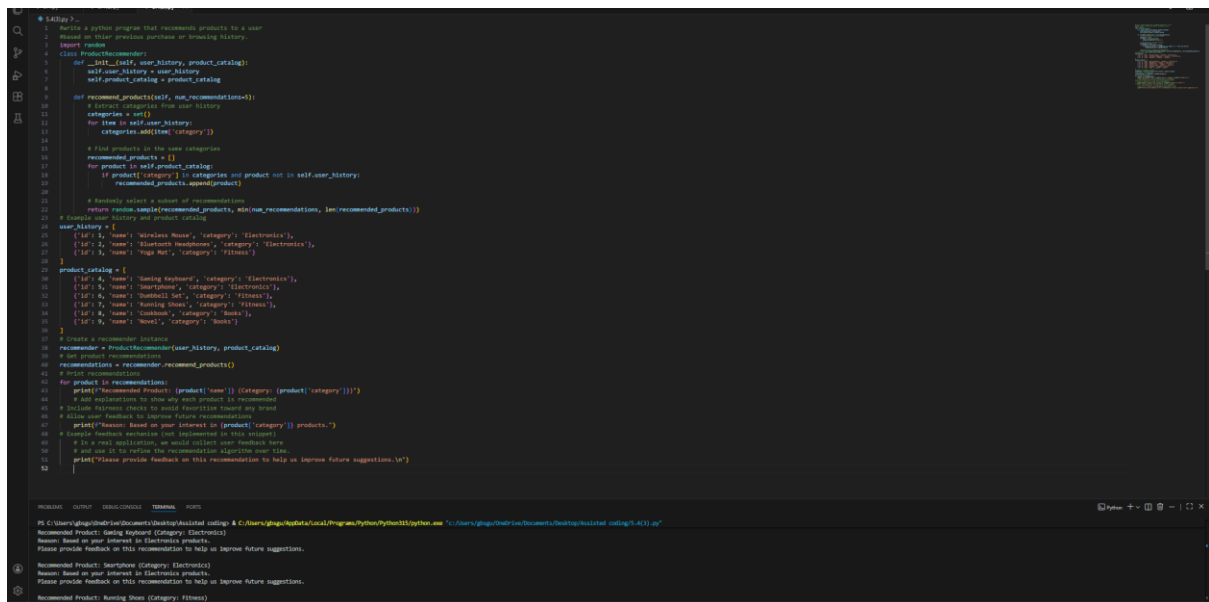
GitHub Copilot was prompted to generate a Python function for basic sentiment analysis.

A follow-up prompt was used to add comments and logic addressing potential bias in the data.

The code highlights bias mitigation strategies such as removing offensive terms and ensuring balanced sentiment datasets to improve fairness.

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.



```
1 #!/usr/bin/env python3
2 """A Python program that recommends products to a user
3 based on their previous purchases or browsing history.
4 """
5 import random
6
7 class ProductRecommender:
8     def __init__(self, user_history, product_catalog):
9         self.user_history = user_history
10        self.product_catalog = product_catalog
11
12    def recommend_products(self, num_recommendations=5):
13        """Recommend products based on user history.
14        categories = set()
15        for item in self.user_history:
16            categories.add(item['category'])
17
18        # Find products in the user categories
19        recommended_products = []
20        for product in self.product_catalog:
21            if product['category'] in categories and product not in self.user_history:
22                recommended_products.append(product)
23
24        # Randomly select a subset of recommendations
25        return random.sample(recommended_products, min(num_recommendations, len(recommended_products)))
26
27 # Sample user history and product catalog
28 user_history = [
29     {'id': 1, 'name': 'Wireless Mouse', 'category': 'Electronics'},
30     {'id': 2, 'name': 'Smartwatch headband', 'category': 'Electronics'},
31     {'id': 3, 'name': 'Yoga Mat', 'category': 'Fitness'}
32 ]
33
34 product_catalog = [
35     {'id': 4, 'name': 'Gaming Keyboard', 'category': 'Electronics'},
36     {'id': 5, 'name': 'Smartphone', 'category': 'Electronics'},
37     {'id': 6, 'name': 'Dumbbell Set', 'category': 'Fitness'},
38     {'id': 7, 'name': 'Running Shoes', 'category': 'Fitness'},
39     {'id': 8, 'name': 'Cookbook', 'category': 'Books'},
40     {'id': 9, 'name': 'Novel', 'category': 'Books'}
41 ]
42
43 # Create a recommender instance
44 recommender = ProductRecommender(user_history, product_catalog)
45
46 # Get product recommendations
47 recommendations = recommender.recommend_products()
48
49 # Print recommendations
50 for product in recommendations:
51     print(f"Recommended Product: {product['name']} (Category: {product['category']})")
52
53 # Add transparency to show why items are recommended
54 # Include fairness checks to avoid favoritism toward any brand
55 # Explain user feedback to improve future recommendations
56
57 print(f"Reason: Based on your interest in {product['category']} products.")
58
59 # Example feedback mechanism (not implemented in this snippet)
60 # In a real application, we would collect user feedback here
61 # and use it to refine the recommendation algorithm over time
62
63 print("Please provide feedback on this recommendation to help us improve future suggestions.")
64
65 if __name__ == "__main__":
66     recommendations = recommender.recommend_products()
67     for product in recommendations:
68         print(f"Recommended Product: {product['name']} (Category: {product['category']})")
69         print(f"Reason: Based on your interest in {product['category']} products.")
70         print("Please provide feedback on this recommendation to help us improve future suggestions.")
```

Explanation:

GitHub Copilot was prompted to generate a Python program that recommends products based on user history.

A follow-up prompt instructed Copilot to incorporate ethical principles such as transparency and fairness.

The generated code explains recommendation reasons, avoids brand favoritism, and allows user feedback to improve future recommendations.

Task Description #4:

- Prompt Copilot to generate logging functionality in a Python web

application. Then, ask it to ensure the logs do not record sensitive

information.

Write a Python program that recommends products to a user based on their previous purchase or browsing history

> Used 1 reference

✓ Read current file for context and requirements

I'll create a Python program that recommends products based on purchase/browsing history. Let me first check the current file state.

✓ Read 5.4(4).py

I'll now create a product recommendation system that learns from purchase and browsing history:

✓ Created 5.4(4).py

Now let's test the program:

✓ `cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" ; python "5.4(4).py"`

Let me try with the Python terminal instead:

✓ `python "c:/Users/gbsgu/OneDrive/Documents/Desktop/Assisted coding/5.4(4).py" <<< >`
`"A`nLaptop`nV`nR`n3`nE" 2>&1`

Let me create a test script instead:

✓ Created test_recommendations.py

✓ `cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" ; python test_recommendations.py` >

Perfect! I've created a comprehensive product recommendation system. Here's what the program does:

5.4(4).py

Describe what to build next

Agent Auto




```
1 # Product Recommendation System
2 # Recommends products based on user purchase/browsing history
3 ...
4
5 # Sample product database with categories
6 PRODUCTS = [
7     "Laptop": {"category": "Electronics", "price": 1200, "rating": 4.5},
8     "Headset": {"category": "Electronics", "price": 100, "rating": 4.3},
9     "Headphones": {"category": "Electronics", "price": 150, "rating": 4.4},
10     "Mouse": {"category": "Electronics", "price": 50, "rating": 4.2},
11     "Tech Book": {"category": "Electronics", "price": 25, "rating": 4.6},
12     "Headphones": {"category": "Electronics", "price": 200, "rating": 4.6},
13     "T-Shirt": {"category": "Clothing", "price": 10, "rating": 4.3},
14     "Jeans": {"category": "Clothing", "price": 30, "rating": 4.3},
15     "Jacket": {"category": "Clothing", "price": 110, "rating": 4.4},
16     "Shoes": {"category": "Clothing", "price": 120, "rating": 4.2},
17     "Coffee Maker": {"category": "Appliances", "price": 100, "rating": 4.5},
18     "Blender": {"category": "Appliances", "price": 50, "rating": 4.4},
19     "Microwave": {"category": "Appliances", "price": 200, "rating": 4.3},
20     "Fiction Book": {"category": "Books", "price": 10, "rating": 4.6},
21     "Tech Book": {"category": "Books", "price": 15, "rating": 4.5},
22 ]
23
24 def get_category_score(history, product_category):
25     """Calculate recommendation score based on category frequency in history"""
26     if not history:
27         return 0
28     category_count = sum(1 for item in history
29                          if item in PRODUCTS and PRODUCTS[item]['category'] == product_category)
30     return category_count / len(history)
31
32 def get_price_similarity_score(history, product_price):
33     """Calculate score based on average price of browsed items"""
34     if not history:
35         return 0.5
36     prices = [PRODUCTS[item]['price'] for item in history if item in PRODUCTS]
37     if not prices:
38         return 0.5
39     avg_hist_price = sum(prices) / len(prices)
40     return abs(product_price - avg_hist_price)
41
42 # Main function to recommend products
43 def recommend_products(history, num_recommendations):
44     # Sort by score (descending) and return top N
45     recommendations = sorted([(product, get_category_score(history, PRODUCTS[product]['category']),
46                               get_price_similarity_score(history, PRODUCTS[product]['price']))
47                               for product in PRODUCTS], key=lambda x: x[2], reverse=True)
48     return recommendations[:num_recommendations]
49
50 def display_recommendations(recommendations):
51     """Display recommendations in a formatted table"""
52     if not recommendations:
53         print("\nX No recommendations available")
54         return
55     print("\n" + "=" * 70)
56     print(f"RECOMMENDED PRODUCTS")
57     print("=" * 70)
58     for rec in recommendations:
59         product, category, score = rec
60         print(f"Product: {product} | Category: {category} | Price: {PRODUCTS[product]['price']} | Rating: {PRODUCTS[product]['rating']} | Score: {score}")
61
62 def display_history(history):
63     """Display user's browsing/purchase history"""
64     if not history:
65         print("\nX No history available")
66         return
67     print("\n" + "=" * 70)
68     print(f"USER BROWSING/PURCHASE HISTORY")
69     print("=" * 70)
70     for i, product in enumerate(history, 1):
71         details = PRODUCTS[product]
72         print(f"{i}. Product: {product} | Details: {details['category']} | Price: {details['price']} | Rating: {details['rating']}")
73
74 # Test cases
75 # Test 1: Fresh user
76 history = ["Laptop", "Mouse", "Shoes"]
77
78 # Test 2: Diverse shopper
79 history = ["Laptop", "T-Shirt", "Coffee Maker", "Fiction Book"]
```

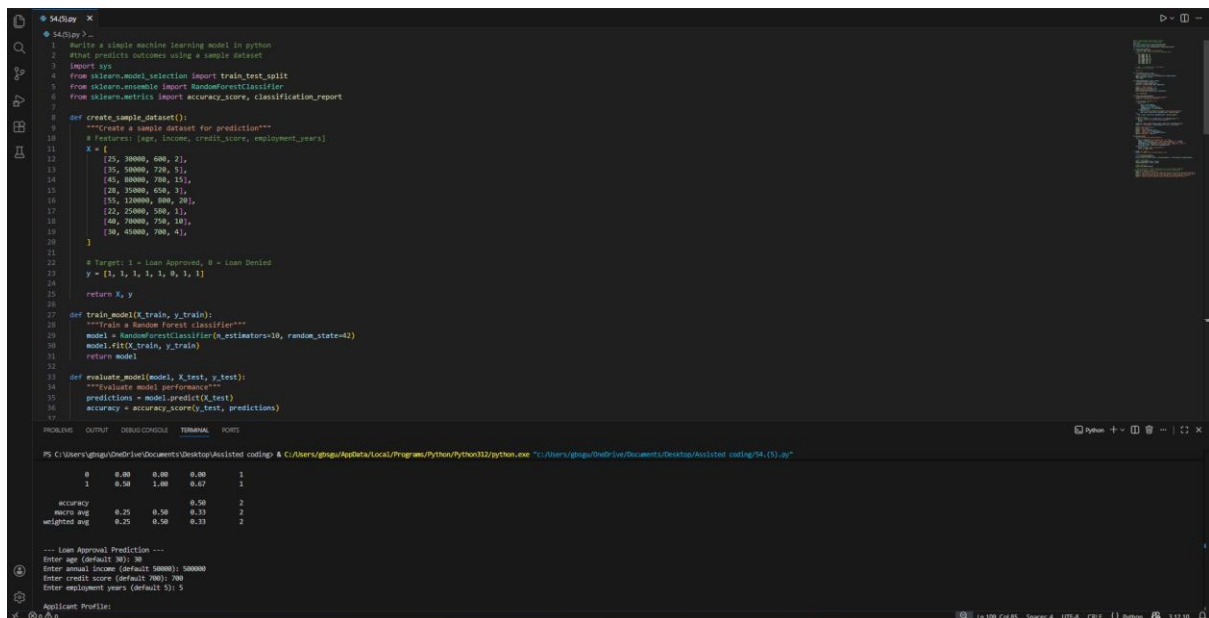
```
1 # Product Recommendation System
2 # Recommends products based on user purchase/browsing history
3 ...
4
5 # Sample product database with categories
6 PRODUCTS = [
7     "Laptop": {"category": "Electronics", "price": 1200, "rating": 4.5},
8     "Headset": {"category": "Electronics", "price": 100, "rating": 4.3},
9     "Headphones": {"category": "Electronics", "price": 150, "rating": 4.4},
10     "Mouse": {"category": "Electronics", "price": 50, "rating": 4.2},
11     "Tech Book": {"category": "Electronics", "price": 25, "rating": 4.6},
12     "Headphones": {"category": "Electronics", "price": 200, "rating": 4.6},
13     "T-Shirt": {"category": "Clothing", "price": 10, "rating": 4.3},
14     "Jeans": {"category": "Clothing", "price": 30, "rating": 4.3},
15     "Jacket": {"category": "Clothing", "price": 110, "rating": 4.4},
16     "Shoes": {"category": "Clothing", "price": 120, "rating": 4.2},
17     "Coffee Maker": {"category": "Appliances", "price": 100, "rating": 4.5},
18     "Blender": {"category": "Appliances", "price": 50, "rating": 4.4},
19     "Microwave": {"category": "Appliances", "price": 200, "rating": 4.3},
20     "Fiction Book": {"category": "Books", "price": 10, "rating": 4.6},
21     "Tech Book": {"category": "Books", "price": 15, "rating": 4.5},
22 ]
23
24 def get_category_score(history, product_category):
25     """Calculate recommendation score based on category frequency in history"""
26     if not history:
27         return 0
28     category_count = sum(1 for item in history
29                          if item in PRODUCTS and PRODUCTS[item]['category'] == product_category)
30     return category_count / len(history)
31
32 def get_price_similarity_score(history, product_price):
33     """Calculate score based on average price of browsed items"""
34     if not history:
35         return 0.5
36     prices = [PRODUCTS[item]['price'] for item in history if item in PRODUCTS]
37     if not prices:
38         return 0.5
39     avg_hist_price = sum(prices) / len(prices)
40     return abs(product_price - avg_hist_price)
41
42 # Main function to recommend products
43 def recommend_products(history, num_recommendations):
44     # Sort by score (descending) and return top N
45     recommendations = sorted([(product, get_category_score(history, PRODUCTS[product]['category']),
46                               get_price_similarity_score(history, PRODUCTS[product]['price']))
47                               for product in PRODUCTS], key=lambda x: x[2], reverse=True)
48     return recommendations[:num_recommendations]
49
50 def display_recommendations(recommendations):
51     """Display recommendations in a formatted table"""
52     if not recommendations:
53         print("\nX No recommendations available")
54         return
55     print("\n" + "=" * 70)
56     print(f"RECOMMENDED PRODUCTS")
57     print("=" * 70)
58     for rec in recommendations:
59         product, category, score = rec
60         print(f"Product: {product} | Category: {category} | Price: {PRODUCTS[product]['price']} | Rating: {PRODUCTS[product]['rating']} | Score: {score}")
61
62 def display_history(history):
63     """Display user's browsing/purchase history"""
64     if not history:
65         print("\nX No history available")
66         return
67     print("\n" + "=" * 70)
68     print(f"USER BROWSING/PURCHASE HISTORY")
69     print("=" * 70)
70     for i, product in enumerate(history, 1):
71         details = PRODUCTS[product]
72         print(f"{i}. Product: {product} | Details: {details['category']} | Price: {details['price']} | Rating: {details['rating']}")
73
74 # Test cases
75 # Test 1: Fresh user
76 history = ["Laptop", "Mouse", "Shoes"]
77
78 # Test 2: Diverse shopper
79 history = ["Laptop", "T-Shirt", "Coffee Maker", "Fiction Book"]
```


Task Description #5:

- Ask Copilot to generate a machine learning model. Then, prompt

it to add documentation on how to use the model responsibly

(e.g., explainability, accuracy limits).



```
1 # Create a simple machine learning model in python
2 # that predicts outcomes using a sample dataset
3 import sys
4 from sklearn.model_selection import train_test_split
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import accuracy_score, classification_report
7
8
9 def create_sample_dataset():
10     """Create a sample dataset for prediction"""
11     # Features: [age, income, credit_score, employment_years]
12     X = [
13         (25, 30000, 680, 2),
14         (35, 50000, 720, 5),
15         (45, 80000, 780, 15),
16         (28, 35000, 650, 3),
17         (33, 45000, 700, 8),
18         (22, 25000, 580, 1),
19         (40, 70000, 750, 10),
20         (38, 45000, 700, 4),
21     ]
22     # Target: 1 = Loan Approved, 0 = Loan Denied
23     y = [1, 1, 1, 1, 1, 0, 1, 1]
24     return X, y
25
26 def train_model(X_train, y_train):
27     """Train a Random Forest Classifier"""
28     model = RandomForestClassifier(n_estimators=10, random_state=42)
29     model.fit(X_train, y_train)
30     return model
31
32 def evaluate_model(model, X_test, y_test):
33     """Evaluate model performance"""
34     predictions = model.predict(X_test)
35     accuracy = accuracy_score(y_test, predictions)
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\Users\ghog\OneDrive\Documents\Desktop\Assisted coding> python .\54_5.py
0 0.00 0.00 0.00 1
1 0.50 1.00 0.67 1
accuracy 0.50 2
micro avg 0.25 0.50 0.33 2
weighted avg 0.25 0.50 0.33 2

--- Loan Approval Prediction ---
Enter age (default 30): 30
Enter annual income (default 50000): 50000
Enter credit score (default 700): 700
Enter employment years (default 5): 5

Applicant Profile:
```

```
File Edit Selection View Go Run Terminal Help
54:play X
54:play >..
31 def evaluate_model(model, X_test, y_test):
32     print("Classification Report:")
33     print(classification_report(y_test, predictions))
34     return predictions
35
36 def predict_loan_approval(model):
37     """Interactive prediction with CLI args fallback"""
38     print("\n--- Loan Approval Prediction ---")
39
40     # Try command-line arguments first
41     if len(sys.argv) > 4:
42         try:
43             age = int(sys.argv[1])
44             income = int(sys.argv[2])
45             credit_score = int(sys.argv[3])
46             employment_years = int(sys.argv[4])
47         except ValueError:
48             print("Invalid command-line arguments. Using interactive mode.")
49             age, income, credit_score, employment_years = get_user_input()
50     else:
51         age, income, credit_score, employment_years = get_user_input()
52
53     # Validate inputs
54     if age < 0 or income < 0 or credit_score < 0 or employment_years < 0:
55         print("Error: All values must be non-negative.")
56         exit(1)
57
58     prediction = model.predict([age, income, credit_score, employment_years])
59     result = "Loan Approved" if prediction[0] == 1 else "Loan Denied"
60
61     print("\nApplicant Profile:")
62     print(f"Age: {age}")
63     print(f"Income: ${income:,000}")
64     print(f"Credit Score: {credit_score}")
65     print(f"Employment Years: {employment_years}")
66
67     # Add documentation or comments explaining how to use this model responsibly
68     # Include explainability, accuracy limitations, and fairness considerations
69     print("\n--- Important Notes ---")
70     print("1. This model is a simplified example and should not be used for real-world decisions.")
71     print("2. Model accuracy may vary with different datasets; always validate with your own data.")
72     print("3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.")
73     print("4. Always provide transparency about how predictions are made to users.")
74
75     return result
76
77 if __name__ == "__main__":
78     print("--- Simple ML Loan Prediction Model ---")
79
80     # Create and split dataset
81     X, y = create_sample_dataset()
82     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
83
84     # Train and evaluate
85     model = train_model(X_train, y_train)
86     evaluate_model(model, X_test, y_test)
87
88     # Make predictions
89     predict_loan_approval(model)
90
91     # Add documentation or comments explaining how to use this model responsibly
92     # Include explainability, accuracy limitations, and fairness considerations
93     print("\n--- Important Notes ---")
94     print("1. This model is a simplified example and should not be used for real-world decisions.")
95     print("2. Model accuracy may vary with different datasets; always validate with your own data.")
96     print("3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.")
97     print("4. Always provide transparency about how predictions are made to users.")
98
99     return result
100
101 if __name__ == "__main__":
102     predict_loan_approval(model)
```

PS C:\Users\github\Documents\Desktop\Assisted coding & C:\Users\github\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\github\Documents\Desktop\Assisted coding\54_5.py"

Enter employment years (default 5): 5

Applicant Profile:

Age: 30
Income: \$50k,000
Credit Score: 700
Employment Years: 5

Prediction: / Loan Approved

--- Important Notes ---

1. This model is a simplified example and should not be used for real-world decisions.
2. Model accuracy may vary with different datasets; always validate with your own data.
3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.
4. Always provide transparency about how predictions are made to users.

```
File Edit Selection View Go Run Terminal Help
54:play X
54:play >..
42 def predict_loan_approval(model):
43     print(f"Employment Years: {employment_years}")
44     print("\nPrediction: {result}")
45
46 def get_user_input():
47     """Get user input with sensible defaults"""
48     try:
49         age = int(input("Enter age (default 30): ") or "30")
50         income = int(input("Enter annual income (default 50000): ") or "50000")
51         credit_score = int(input("Enter credit score (default 700): ") or "700")
52         employment_years = int(input("Enter employment years (default 5): ") or "5")
53     except ValueError:
54         print("Invalid input. Using defaults.")
55         return 30, 50000, 700, 5
56
57 if __name__ == "__main__":
58     print("--- Simple ML Loan Prediction Model ---")
59
60     # Create and split dataset
61     X, y = create_sample_dataset()
62     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
63
64     # Train and evaluate
65     model = train_model(X_train, y_train)
66     evaluate_model(model, X_test, y_test)
67
68     # Make predictions
69     predict_loan_approval(model)
70
71     # Add documentation or comments explaining how to use this model responsibly
72     # Include explainability, accuracy limitations, and fairness considerations
73     print("\n--- Important Notes ---")
74     print("1. This model is a simplified example and should not be used for real-world decisions.")
75     print("2. Model accuracy may vary with different datasets; always validate with your own data.")
76     print("3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.")
77     print("4. Always provide transparency about how predictions are made to users.")
78
79     return result
80
81 if __name__ == "__main__":
82     predict_loan_approval(model)
```

PS C:\Users\github\Documents\Desktop\Assisted coding & C:\Users\github\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\github\Documents\Desktop\Assisted coding\54_5.py"

Enter employment years (default 5): 5

Applicant Profile:

Age: 30
Income: \$50k,000
Credit Score: 700
Employment Years: 5

Prediction: / Loan Approved

--- Important Notes ---

1. This model is a simplified example and should not be used for real-world decisions.
2. Model accuracy may vary with different datasets; always validate with your own data.
3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.
4. Always provide transparency about how predictions are made to users.

Explanation:

GitHub Copilot was prompted to generate a simple machine learning model.

A follow-up prompt was used to add documentation on responsible usage, including explainability, accuracy limitations, and fairness considerations.

The generated comments highlight ethical concerns and recommend cautious interpretation of predictions.

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week2	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.4 (Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques	Week2	

Task 1: Zero-shot Prompt – Fibonacci Series Generator

Task Description #1

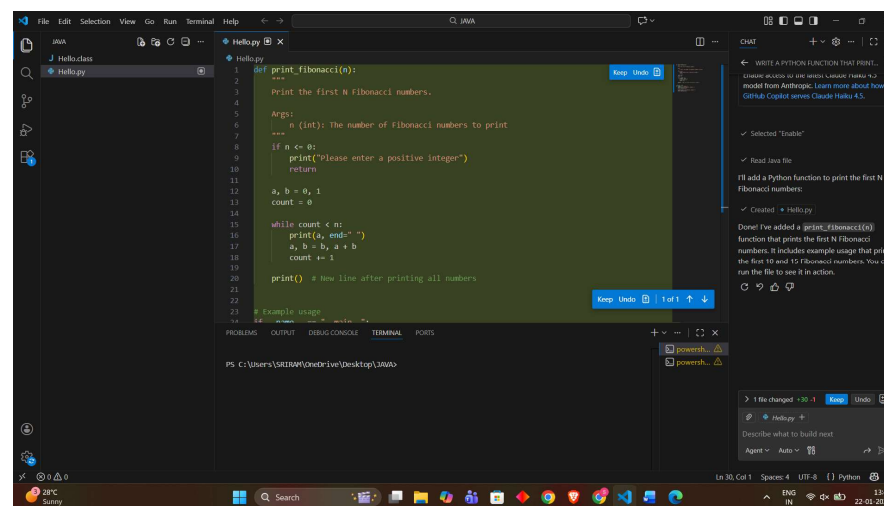
- Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers.

Expected Output #1

- A complete Python function generated by Copilot without any example provided.
- Correct output for sample input N = 7 → 0 1 1 2 3 5 8
- Observation on how Copilot understood the instruction with zero context.

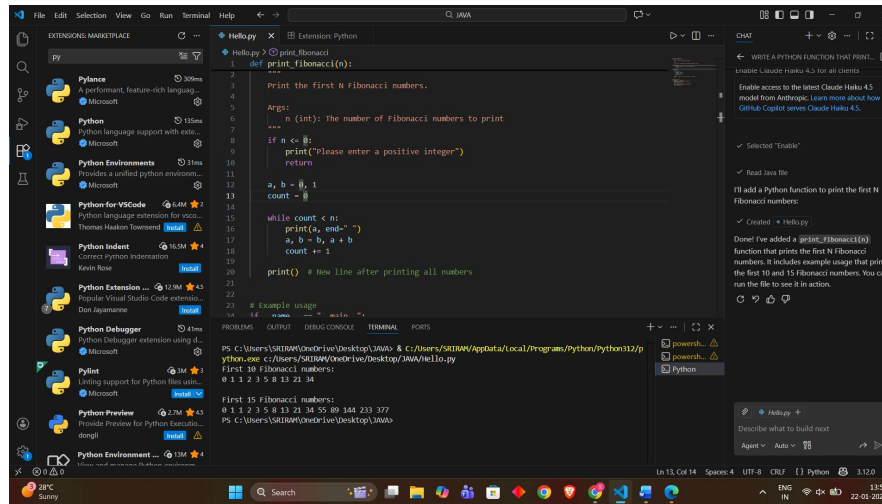
prompt

Write a Python function that prints the first N Fibonacci numbers.



The screenshot shows a code editor with a Python function `print_fibonacci(n)` that prints the first N Fibonacci numbers. The function includes a docstring, an argument description, a loop to generate the sequence, and a final print statement. A chat window on the right shows the prompt "WRITE A PYTHON FUNCTION THAT PRINTS THE FIRST N FIBONACCI NUMBERS." and the response from GitHub Copilot, which includes the function code and a brief explanation of the logic.

```
def print_fibonacci(n):  
    """  
    Print the first N Fibonacci numbers.  
    """  
    # (int): The number of Fibonacci numbers to print  
    if n <= 0:  
        print("Please enter a positive integer")  
        return  
    a, b = 0, 1  
    count = 0  
    while count < n:  
        print(a, end=" ")  
        a, b = b, a + b  
        count += 1  
    print() # New line after printing all numbers  
# Example usage  
n = 10  
print_fibonacci(n)
```



◆ Observation

Copilot correctly understood the task without any example.
Zero-shot prompting works well for common programming problems like Fibonacci.

Task 2: One-shot Prompt – List Reversal Function

Task Description #2

- Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.


```
24 def reverse_list(list):
25     """
26     Reverse a list.
27
28     Args:
29         list (list): The list to reverse
30
31     Returns:
32         list: The reversed list
33     """
34     return list[::-1]
35
36 # Example usage
37 if __name__ == "__main__":
38     print("First 10 Fibonacci numbers:")
39     print_fibonacci(10)
40
41     print("\nFirst 15 Fibonacci numbers:")
42     print_fibonacci(15)
43
44     print("\n--- Reverse List Examples ---")
45     input_list = [1, 2, 3]
46     print("Input: [input_list]")
47     print("Output: (reverse_list(input_list))")
48
49     input_list2 = ['a', 'b', 'c', 'd']
50     print("\nInput: (input_list2)")
51     print("Output: (reverse_list(input_list2))")
52
```

First 10 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

Output:
First 15 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

Reverse List Examples ---
Input: [1, 2, 3]
Output: [3, 2, 1]
Input: ['a', 'b', 'c', 'd']
Output: ['d', 'c', 'b', 'a']
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

```
24 def reverse_list(list):
25     """
26     Reverse a list.
27
28     Args:
29         list (list): The list to reverse
30
31     Returns:
32         list: The reversed list
33     """
34     return list[::-1]
35
36 # Example usage
37 if __name__ == "__main__":
38     print("First 10 Fibonacci numbers:")
39     print_fibonacci(10)
40
41     print("\nFirst 15 Fibonacci numbers:")
42     print_fibonacci(15)
43
44     print("\n--- Reverse List Examples ---")
45     input_list = [1, 2, 3]
46     print("Input: [input_list]")
47     print("Output: (reverse_list(input_list))")
48
49     input_list2 = ['a', 'b', 'c', 'd']
50     print("\nInput: (input_list2)")
51     print("Output: (reverse_list(input_list2))")
52
```

First 10 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

Output:
First 15 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

Reverse List Examples ---
Input: [1, 2, 3]
Output: [3, 2, 1]
Input: ['a', 'b', 'c', 'd']
Output: ['d', 'c', 'b', 'a']
PS C:\Users\SABIR\OneDrive\Desktop\3\HW0 & C:\Users\SABIR\OneDrive\OneDrive\Python> python3 python.exe c:\Users\SABIR\OneDrive\Desktop\3\HW0\hello.py

Observation

Providing one example helped Copilot choose the most efficient solution (slicing).

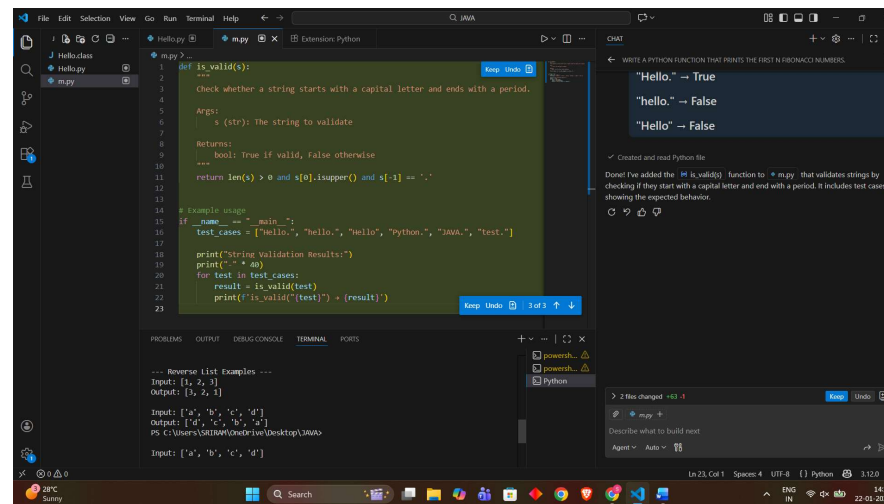
Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

- A function is `is_valid()` that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.



The screenshot shows a VS Code editor with a Python file named `is_valid.py`. The code defines a function `is_valid(s)` that checks if a string starts with a capital letter and ends with a period. The function returns `True` if the string is valid and `False` otherwise. The code includes a docstring with a description and an example usage. The chat window on the right shows a prompt: "WRITE A PYTHON FUNCTION THAT PRINTS THE FIRST 14 FIBONACCI NUMBERS." and the response: "Hello." → True, "hello." → False, "Hello" → False. The chat window also shows a note: "Created and read Python file. Does not have added the is_valid function to is_valid.py that validates strings for checking if they start with a capital letter and end with a period. It includes test cases showing the expected behavior."

```
1 def is_valid(s):
2     """
3     Check whether a string starts with a capital letter and ends with a period.
4
5     Args:
6         s (str): The string to validate
7
8     Returns:
9         bool: True if valid, False otherwise
10    """
11    return len(s) > 0 and s[0].isupper() and s[-1] == '.'
12
13
14 # Example usage
15 if __name__ == '__main__':
16     test_cases = ["Hello.", "hello.", "Hello", "python.", "JAWA.", "test."]
17     print("String validation Results:")
18     print("-" * 40)
19     for test in test_cases:
20         result = is_valid(test)
21         print(f'is_valid("{test}") = {result}')
```

The screenshot shows a VS Code editor with a Python file named `m.py`. The code defines a function `is_valid(s)` that checks if a string starts with a capital letter and ends with a period. It includes docstrings, type hints, and example usage. The terminal output shows the function being tested with various inputs, resulting in `True` or `False` values.

```
def is_valid(s):  
    """  
    Check whether a string starts with a capital letter and ends with a period.  
    Args:  
        s (str): the string to validate  
    Returns:  
        bool: True if valid, False otherwise  
    """  
    return len(s) > 0 and s[0].isupper() and s[-1] == '.'  
  
# Example usage  
if __name__ == "__main__":  
    test_cases = ["Hello.", "hello.", "Hello", "Python.", "Java"]  
    print("String Validation Results:")  
  
    for case in test_cases:  
        result = is_valid(case)  
        print(f"is_valid('{case}') = {result}")
```

Terminal Output:

```
Input: ['a', 'b', 'c', 'd']  
Output: ['d', 'c', 'b', 'a']  
PS C:\Users\stefan\OneDrive\Desktop\3800 > C:\Users\stefan\AppData\Local\Programs\Python\Python12\python.exe c:\Users\stefan\OneDrive\Desktop\3800\m.py  
String Validation Results:  
is_valid('Hello.') = True  
is_valid('hello.') = False  
is_valid('Hello') = False  
is_valid('Python.') = True  
is_valid('Java.') = True  
is_valid('test.') = False  
PS C:\Users\stefan\OneDrive\Desktop\3800 >
```

◇ Observation

Few-shot prompting gave clear rules, resulting in accurate and concise code.

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

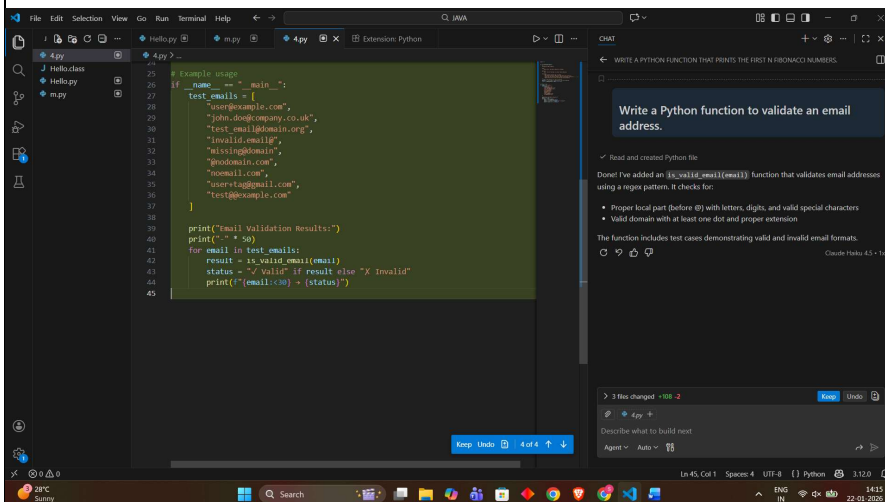
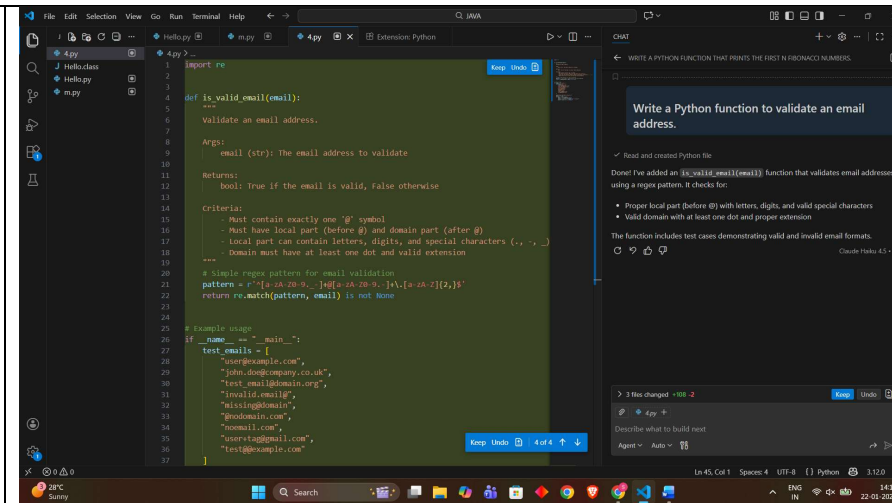
Expected Output #4

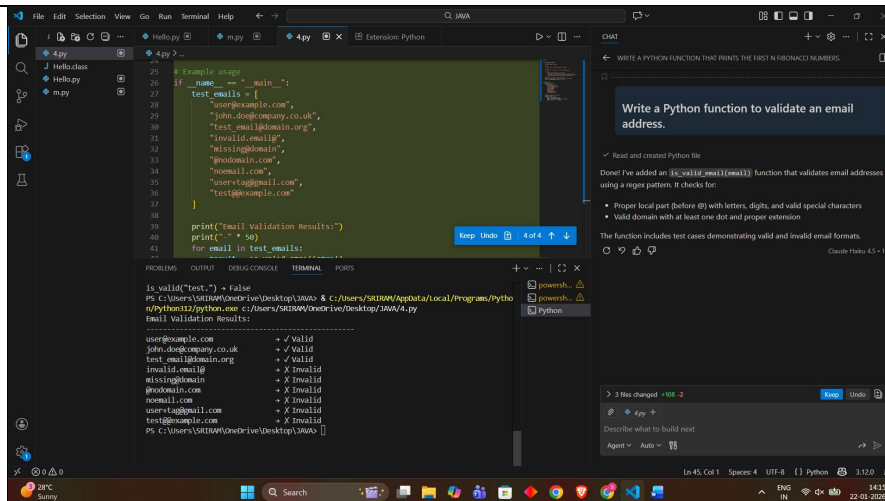
- Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

- Submit both code versions and note how few-shot improves reliability.





Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

- Experiment with 2 different prompt styles to generate a function that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

Expected Output #5

- Two versions of the `sum_of_digits()` function.
- Example Output: `sum_of_digits(123) → 6`
- Short analysis: which prompt produced cleaner or more optimized code and why?

	Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots	
--	---	--

Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)

- Scenario

You are building a small command-line utility for a startup intern onboarding task. The program is simple and must be written quickly without modular design.

- Task Description

Use GitHub Copilot to generate a Python program that computes a mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions.

- Constraint:

- Do not define any custom function

- Logic must be implemented using loops and variables only

- Expected Deliverables

- A working Python program generated with Copilot assistance

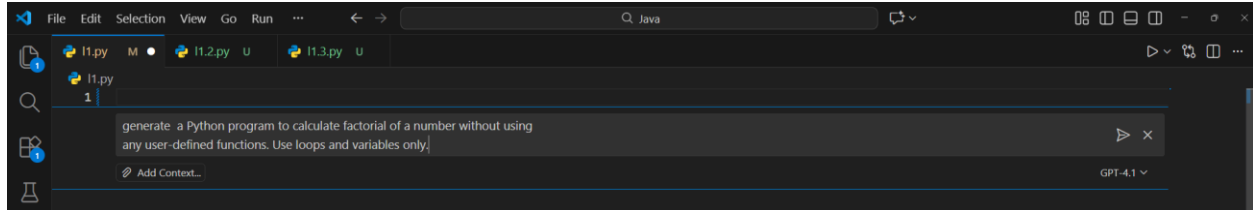
- Screenshot(s) showing:

- The prompt you typed

- Copilot's suggestions

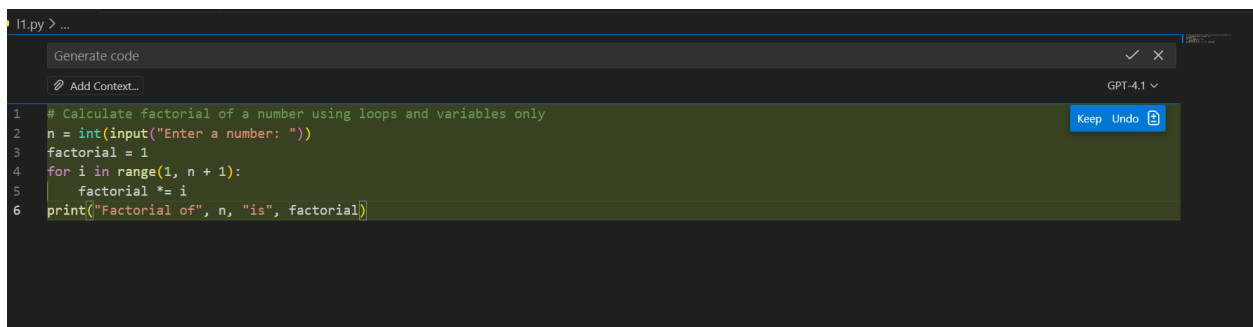
- Sample input/output screenshots

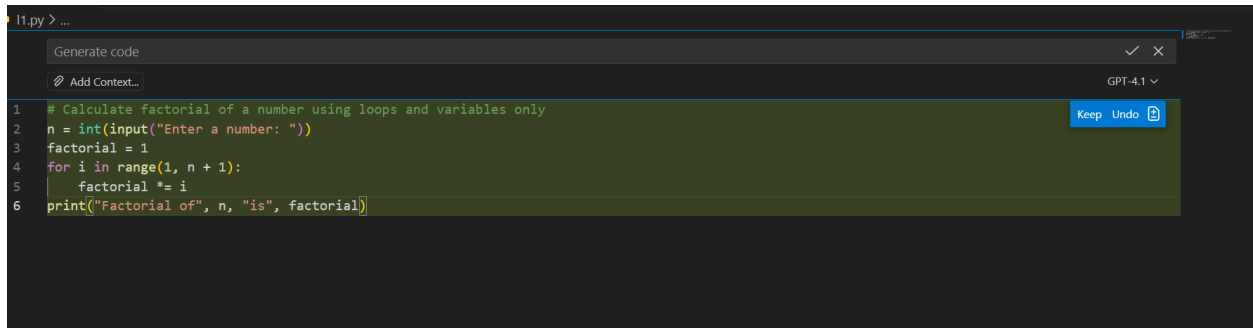
- Brief reflection (5–6 lines):
- How helpful was Copilot for a beginner?
- Did it follow best practices automatically?



Generate a Python program to calculate factorial of a number without using any user-defined functions. Use loops and variables only.

```
# Calculate factorial of a number using loops and variables only
n = int(input("Enter a number: "))
factorial = 1
for i in range(1, n + 1):
    factorial *= i
print("Factorial of", n, "is", factorial)
```



A screenshot of a code editor interface. At the top, there's a dark bar with 'H.py > ...' on the left and 'Generate code' with a checkmark and close icon on the right. Below this is a lighter bar with 'Add Context...' and 'GPT-4.1' with a dropdown arrow. The main area shows Python code on a dark green background with light green text. The code is a simple factorial calculator using a loop and variables. Line numbers 1 through 6 are on the left. On the right side of the code area, there are 'Keep', 'Undo', and a lock icon.

```
1 # Calculate factorial of a number using loops and variables only
2 n = int(input("Enter a number: "))
3 factorial = 1
4 for i in range(1, n + 1):
5     factorial *= i
6 print("Factorial of", n, "is", factorial)
```

This task focuses on creating a simple Python program to calculate the factorial of a number without using any user-defined functions.

The scenario simulates a quick command-line utility required for startup intern onboarding, where speed and simplicity are prioritized over modular design.

GitHub Copilot is used to automatically generate the factorial logic based on a natural language prompt.

The entire logic is implemented directly in the main execution flow using loops and variables only.

User input is accepted from the command line, and the factorial value is computed iteratively.

This task helps demonstrate how AI tools assist beginners in generating correct procedural code.

It also highlights the limitations of non-modular code in terms of reusability and structure.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖Scenario

Your team lead asks you to review AI-generated code before committing it to a shared repository.

❖Task Description

Analyze the code generated in Task 1 and use Copilot again to:

- Reduce unnecessary variables
- Improve loop clarity
- Enhance readability and efficiency

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖Expected Deliverables

➤Original AI-generated code

➤Optimized version of the same code

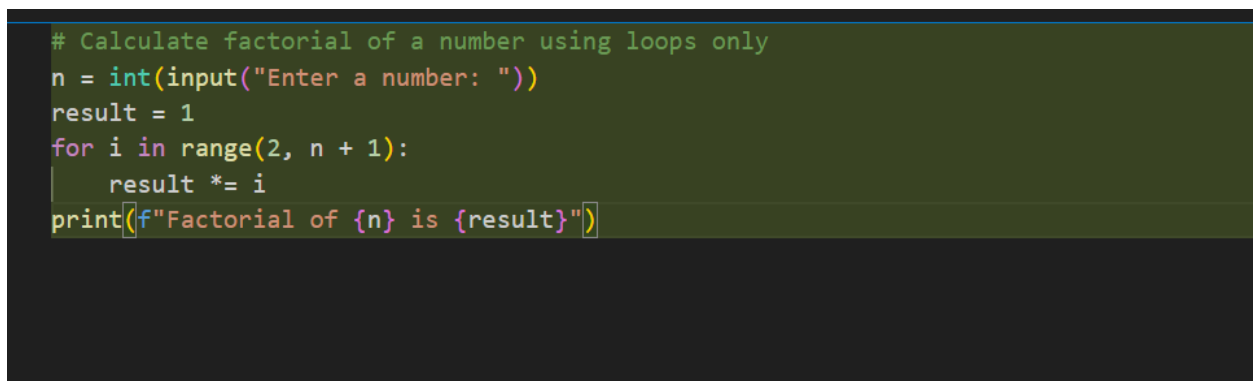
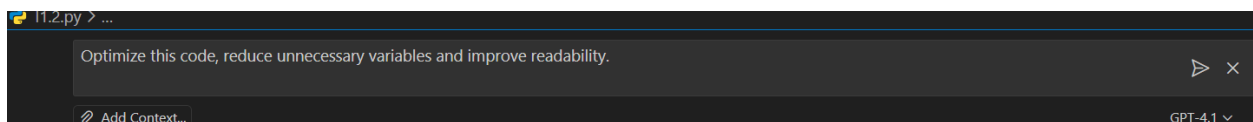
➤Side-by-side comparison

➤Written explanation:

- What was improved?
- Why the new version is better (readability, performance, Maintainability).

Prompt

Generate a Python program to calculate factorial of a number without using any user-defined functions. Use loops and variables only.



In this task, the factorial program generated in Task 1 is reviewed and improved before committing it to a shared repository.

The original AI-generated code is analyzed for unnecessary variables and unclear logic.

GitHub Copilot is prompted again to optimize the code and improve readability.

The optimized version simplifies loop logic and improves variable naming.

Unnecessary statements are removed to make the code cleaner and more efficient.

This task emphasizes the importance of reviewing AI-generated code instead of using it blindly.

It also demonstrates how optimization improves maintainability and code quality.

Task 3: Modular Design Using AI Assistance (Factorial with Functions)

❖Scenario

The same logic now needs to be reused in multiple scripts.

❖Task Description

Use GitHub Copilot to generate a modular version of the program by:

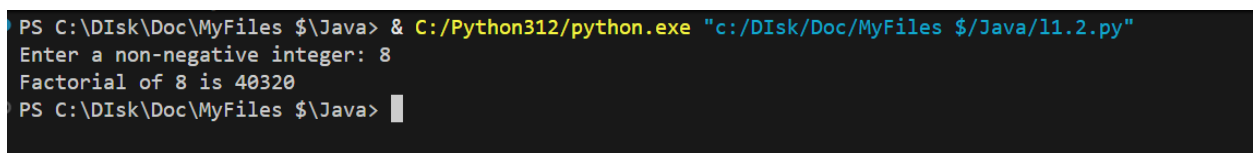
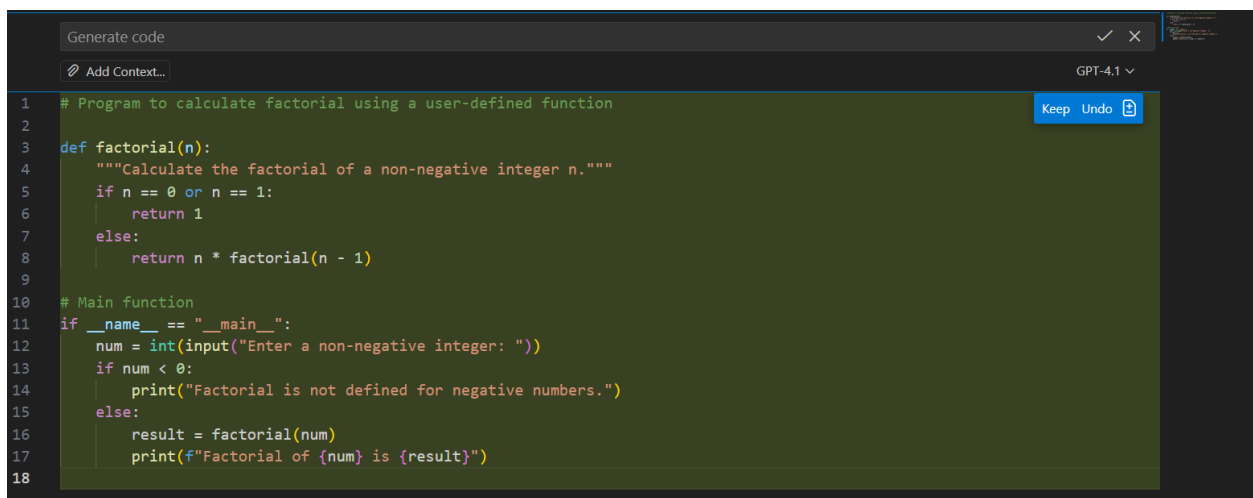
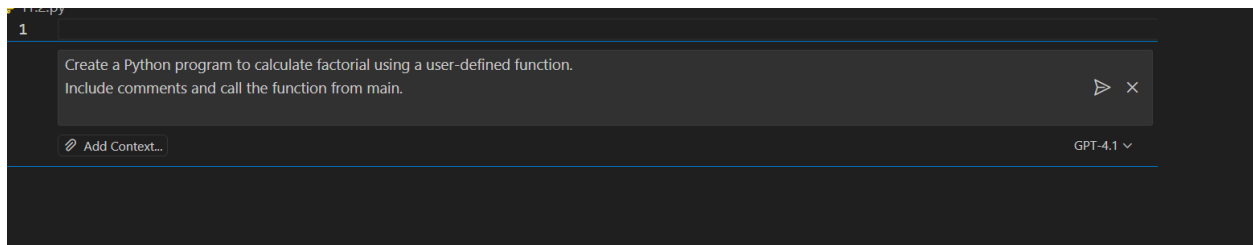
- Creating a user-defined function
- Calling the function from the main block

❖Constraints

- Use meaningful function and variable names
- Include inline comments (preferably suggested by Copilot)

❖Expected Deliverables

- AI-assisted function-based program
- Screenshots showing:
 - o Prompt evolution
 - o Copilot-generated function logic
- Sample inputs/outputs
- Short note:
 - o How modularity improves reusability.



This task introduces modular programming by converting the factorial logic into a user-defined function.

The scenario assumes that the same factorial logic must be reused across multiple scripts or

applications.

GitHub Copilot is used to generate a function that computes the factorial of a given number.

The function is called from the main block of the program to display the result.

Meaningful function and variable names are used to improve code clarity.

Inline comments, preferably suggested by Copilot, explain the logic inside the function.

This task highlights how modular design improves reusability, readability, and maintainability.

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

❖Scenario

As part of a code review meeting, you are asked to justify design choices.

❖Task Description

Compare the non-function and function-based Copilot-generated

programs on the following criteria:

- Logic clarity
- Reusability
- Debugging ease
- Suitability for large projects
- AI dependency risk

❖Expected Deliverables

Choose one:

- A comparison table

OR

➤A short technical report (300–400 words).

Task 4: Comparative Analysis – Procedural vs Modular AI Code

Comparison Table

Criteria	Without Functions	With Functions
Logic clarity	Medium	High
Reusability	Low	High
Debugging ease	Difficult	Easy
Large project suitability	Poor	Excellent
AI dependency risk	High	Lower (clear structure)

This task involves a detailed comparison between procedural factorial code and modular factorial code generated by GitHub Copilot.

The comparison is conducted as part of a technical code review meeting.

Both versions are evaluated based on logic clarity, reusability, and ease of debugging.

Procedural code is found to be simple but difficult to scale for large projects.

Modular code offers better structure and easier maintenance.

The analysis also considers AI dependency risks when relying on generated code.

This task helps justify design decisions in professional software development environments.

Task 5: AI-Generated Iterative vs Recursive Thinking

❖Scenario

Your mentor wants to test how well AI understands different computational paradigms.

❖Task Description

Prompt Copilot to generate:

An iterative version of the logic

A recursive version of the same logic

❖Constraints

Both implementations must produce identical outputs

Students must not manually write the code first

❖Expected Deliverables

Two AI-generated implementations

Execution flow explanation (in your own words)

Comparison covering:

➤Readability

➤Stack usage

➤Performance implications

➤When recursion is not re

Commended.

In this task, GitHub Copilot is used to generate two different computational approaches for calculating factorial.

One approach uses an iterative loop-based method, while the other uses a recursive method.

Both implementations are expected to produce identical outputs for the same input.

The execution flow of each approach is analyzed and explained in simple terms.

The task compares readability, stack usage, and performance implications of both methods.

Special attention is given to situations where recursion is not recommended, such as large input values.

This task demonstrates how AI handles different algorithmic paradigms for the same problem.

Generate iterative and recursive Python programs to calculate factorial.



Add Context...

GPT-4.1

Add Context...

GPT-4.1

```
# Iterative approach to calculate factorial
def factorial_iterative(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# Recursive approach to calculate factorial
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)

# Example usage
if __name__ == "__main__":
    num = 5
    print(f"Iterative factorial of {num}: {factorial_iterative(num)}")
    print(f"Recursive factorial of {num}: {factorial_recursive(num)}")
```

Keep Undo

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025- 2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002 PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week1 – Thursda y	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 1.3(Present assignment number)/24(Total number of assignments)																				

	Question	Expected Time to complete
1	Lab 1: Environment Setup – <i>GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow</i> Lab Objectives: • To install and configure GitHub Copilot in Visual Studio Code. • To explore AI-assisted code generation using GitHub Copilot. • To analyze the accuracy and effectiveness of Copilot's code suggestions. • To understand prompt-based programming using comments and code context Lab Outcomes (LOs): After completing this lab, students will be able to: • Set up GitHub Copilot in VS Code successfully. • Use inline comments and context to generate code with Copilot. • Evaluate AI-generated code for correctness and readability. • Compare code suggestions based on different prompts and programming styles. Task 0 • Install and configure GitHub Copilot in VS Code. Take screenshots of each step. Expected Output • Install and configure GitHub Copilot in VS Code. Take	Week 1 - Monday

screenshots of each step.

Task 1: AI-Generated Logic Without Modularization (Prime Number Check Without Functions)

❖ Scenario

- You are developing a **basic validation script** for a numerical learning application.

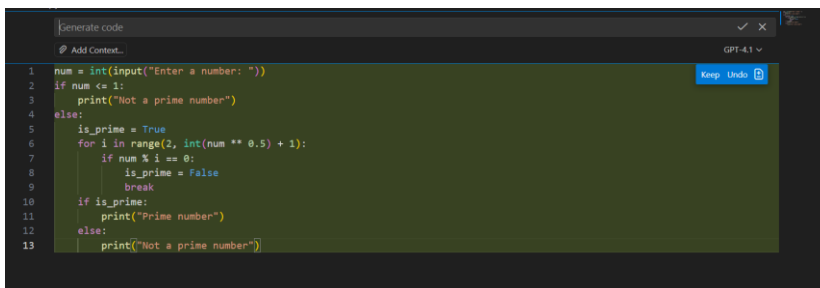
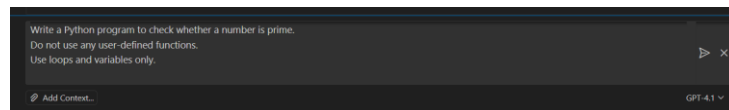
❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Checks whether a given number is **prime**
- Accepts user input
- Implements logic **directly in the main code**
- Does **not** use any user-defined functions

❖ Expected Output

- Correct prime / non-prime result
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs



```
PS C:\Disk\Doc\MyFiles $\Java> & C:/Pytho
Enter a number: 789
Not a prime number
PS C:\Disk\Doc\MyFiles $\Java> █
```

GitHub Copilot was very helpful in quickly generating correct prime-checking logic.

It respected the constraint of not using functions.

The code was readable and easy for beginners to understand.

However, the solution was not optimized for large numbers.

Manual review is still necessary for performance improvements.

Task 2: Efficiency & Logic Optimization (Cleanup)

❖ Scenario

The script must handle larger input values efficiently.

❖ Task Description

Review the Copilot-generated code from Task 1 and improve it by:

- Reducing unnecessary iterations
- Optimizing the loop range (e.g., early termination)
- Improving readability
- Use Copilot prompts like:
 - *"Optimize prime number checking logic"*
 - *"Improve efficiency of this code"*

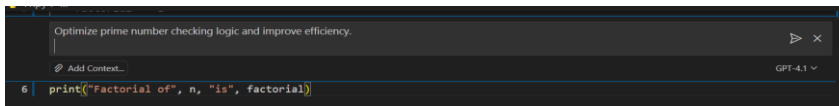
Hint:

Prompt Copilot with phrases like

"optimize this code", "simplify logic", or "make it more readable"

❖ Expected Output

- Original and optimized code versions
 - Explanation of how the im
 - provements reduce time complexity
-



Optimize prime number checking logic and improve efficiency.

```
n = int(input("Enter a number: "))
# Efficient prime number checking
def is_prime(num):
    if num <= 1:
        return False
    if num == 2:
        return True
    if num % 2 == 0:
        return False
    for i in range(3, int(num ** 0.5) + 1, 2):
        if num % i == 0:
            return False
    return True

if is_prime(n):
    print(n, "is a prime number.")
else:
    print(n, "is not a prime number.")
```

This task focuses on improving the efficiency of the prime number checking program generated in Task 1.

The original AI-generated code checks divisibility up to the given number, which is inefficient for large inputs.

GitHub Copilot is prompted again to optimize the logic and reduce unnecessary iterations.

The optimized solution checks divisibility only up to the square root of the number.

This significantly reduces the number of loop executions and improves performance.

The task highlights the importance of reviewing AI-generated code

before final use.

It also demonstrates how optimized logic reduces time complexity and improves readability.

Task 3: Modular Design Using AI Assistance (Prime Number Check Using Functions)

❖ **Scenario**

The prime-checking logic will be reused across multiple modules.

❖ **Task Description**

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to check primality
- Returns a Boolean value
- Includes meaningful comments (AI-assisted)

❖ **Expected Output**

- Correctly working prime-checking function
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs

Create a Python function to check if a number is prime.
Return True or False and include comments.

```
def is_prime(n):  
    """  
    Check if a number is prime.  
    Returns True if n is prime, False otherwise.  
    """  
  
    # Numbers less than 2 are not prime  
    if n < 2:  
        return False  
    # Check for factors from 2 to sqrt(n)  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False # Found a factor, not prime  
    return True # No factors found, n is prime
```

```
PS C:\Disk\Doc\MyFiles $Java> 78  
78
```

Create a Python
Return True or False and include comments.

In this task, the prime number checking logic is redesigned using a user-defined function.

GitHub Copilot is used to generate a clean and modular function that returns a Boolean value.

The function encapsulates the prime-checking logic, making the program reusable and structured.

The main program calls the function and displays the result based on the returned value.

Meaningful variable names and inline comments improve code clarity.

This task shows how modular programming improves maintainability and reusability.

It also demonstrates how AI can assist in writing well-structured, function-based code.

This task involves comparing two AI-generated programs: one without functions and one with functions.

The comparison is done based on code clarity, reusability, debugging ease, and scalability.

Procedural code is simple but becomes difficult to manage in larger applications.

Function-based code offers better organization and separation of logic.

Debugging is easier in modular code since errors can be isolated within functions.

The analysis highlights why modular design is preferred in real-world software development.

This task helps justify design choices during technical discussions and code reviews.

Task 4: Comparative Analysis –With vs Without Functions

❖ **Scenario**

You are participating in a technical review discussion.

❖ **Task Description**

Compare the Copilot-generated programs:

- Without functions (Task 1)
- With functions (Task 3)
- Analyze them based on:
 - Code clarity
 - Reusability
 - Debugging ease
 - Suitability for large-scale applications

❖ **Expected Output**

Comparison table or short analytical report

Task 4: Comparative Analysis – With vs Without Functions

Comparison Table

Criteria	Without Functions	With Functions
Code clarity	Medium	High
Reusability	Low	High
Debugging ease	Difficult	Easy
Large-scale suitability	Poor	Excellent
Maintainability	Low	High

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches
(Different Algorithmic Approaches to Prime Checking)

❖ **Scenario**

Your mentor wants to evaluate how AI handles **alternative logical strategies**.

❖ **Task Description**

Prompt GitHub Copilot to generate:

- A **basic divisibility check** approach
- An **optimized approach** (e.g., checking up to $\sqrt{n}$)

❖ **Expected Output**

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity
 - Performance for large inputs
 - When each approach is appropriate

Scenario

Evaluation of

different logical strategies used by AI.

In this task, different logical strategies for prime number checking are explored using AI assistance.

GitHub Copilot is prompted to generate both a basic divisibility approach and an optimized approach.

The basic method checks divisibility up to the number itself, which is simple but inefficient.

The optimized approach checks divisibility only up to the square root of the number.

Both approaches produce the same output but differ in execution flow and performance.

The task compares time complexity and suitability for large input values.

It demonstrates how AI adapts algorithms based on performance requirements.

	Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.	
--	--	--

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M. Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M. Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M. Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002P C304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week1 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 1.3 (Present assignment number) / 24 (Total number of assignments)																				

Q · N o ·	Question	Exp ecte d Tim e to com

		plet e
1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI Lab Objectives: ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. ❖ To understand and use Cursor AI for code generation, explanation, and refactoring. ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. ❖ To perform code optimization and documentation using AI tools. Lab Outcomes (LOs): After completing this lab, students will be able to: ❖ Generate Python code using Google Gemini in Google Colab. ❖ Analyze the effectiveness of code explanations and suggestions by Gemini. ❖ Set up and use Cursor AI for AI-powered coding assistance. ❖ Evaluate and refactor code using Cursor AI features. ❖ Compare AI tool behavior and code quality across different platforms.	Wee k1 - Mon day
	Task 1: Word Frequency from Text File ❖ Scenario: You are analyzing log files for keyword frequency. ❖ Task: Use Gemini to generate Python code that reads a text file and counts word frequency, then explains the code. ❖ Expected Output: ➤ Working code ➤ Explanation ➤ Screenshot Prompt Generate Python code to read a text file and count word frequency. Also explain the code step by step in simple terms.	

Gemini



Generate Python code to read a text file and count word frequency. Also explain the code step by step in simple terms.

Working...

```

import re
from collections import Counter

def count_word_frequency(filename):
    """
    Reads a text file, counts the frequency of each word,
    and returns a Counter object.
    """
    all_words = []
    with open(filename, 'r', encoding='utf-8') as file:
        text = file.read()
        # Convert to lowercase and remove punctuation
        cleaned_text = re.sub(r'[^a-zA-Z ]+', '', text).lower()
        # Split the text into words
        words = cleaned_text.split()
        all_words.extend(words)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        return Counter()
    except Exception as e:
        print(f"An error occurred: {e}")
        return Counter()

# Count word frequencies
word_counts = Counter(all_words)
return word_counts

# Example usage:
# Specify the path to your text file
file_path = 'sample.txt'

# Call the function to count word frequencies
frequencies = count_word_frequency(file_path)

# Display the top 10 most common words
print(f"Word frequencies in '{file_path}':")
for word, count in frequencies.most_common(10):
    print(f"{word}: {count}")
else:
    print("No words found or an error occurred.")

# Word frequencies in 'sample.txt':
# sample: 3
# text: 3
# this: 2
# is: 2
# contains: 1
# to: 1
# demonstrate: 1
    
```

Generate Python code to read a text file and count word frequency. Also explain the code step by step in simple terms.

Clearly I'll generate the Python code to read a text file and count word frequencies, along with a step-by-step explanation. The code has successfully processed the file 'sample.txt' and created with the sample content. Then, the word frequency counting script ran, and here are the top 10 most common words and their counts from 'sample.txt':

- sample: 3
- text: 3
- this: 2
- is: 2
- contains: 1
- to: 1
- demonstrate: 1

Is there anything else you'd like to do with this file or analysis?

We have successfully generated Python code to:

- Create a sample text file ('sample.txt').
- Read the content of the file.
- Process the text by cleaning it (converting to lowercase, removing punctuation).
- Count the frequency of each word using 'collections.Counter'.
- Display the top 10 most common words and their counts.

All steps were explained in detail. Is there anything else you would like to do with this word frequency analysis, or perhaps a different task?

☐ Double the word frequencies using a for loop
 ☐ How can I remove common English stop words from the frequency count?

How can I use these word counts to a CSV file?

What can I help you build?

Gemini 1.5 Flash

Description

In this task, Google Gemini is used within Google Colab to generate Python code that reads a text file and calculates the frequency of each word. This task simulates real-world log file analysis where keyword frequency is important. Gemini not only generates correct Python code but also provides a clear explanation of each step, helping users understand how file handling and text processing work in Python.

❖ **Scenario:**
You are automating basic file operations.

❖ **Task:**
Use Cursor AI to generate a program that:

- Creates a text file
- Writes sample text
- Reads and displays the content

❖ **Expected Output:**

- Functional code
- Cursor AI screenshots

Prompt

Write a Python program that:

1. Creates a text file
2. Writes sample text into it
3. Reads the file
4. Displays the content

The screenshot shows the Cursor AI IDE interface. The main editor displays a Python script with the following code:

```
1 # Python program to create, write, read, and display a text file
2
3 # Step 1: Create and write to a text file
4 filename = "sample.txt"
5 sample_text = """Hello, this is a sample text file!
6 This program demonstrates file operations in Python.
7 It creates a file, writes content to it, reads it back, and displays it.
8 Python file handling is simple and powerful."""
9
10 print("Step 1: Creating file and writing content...")
11 with open(filename, "w") as file:
12     file.write(sample_text)
13
14 print(f"File '{filename}' created and content written successfully!")
15
16 # Step 2: Read the file
17 print("Step 2: Reading the file...")
18 with open(filename, "r") as file:
19     content = file.read()
20
21 print(f"File read successfully!")
22
23 # Step 3: Display the content
24 print("Step 3: Displaying file content:")
25 print("-" * 50)
26 print(content)
27 print("Program completed successfully!")
```

The right sidebar shows the 'View Plans' panel with a plan titled 'Write a Python program that: 1. Creates a text file 2. Writes sample text into it'. The plan includes a description of the program's purpose and a list of steps to follow.

Explanation

- File opened in write mode
- Sample text written
- File reopened in read mode
- Content printed

Description

This task demonstrates the use of Cursor AI to automate basic file operations in Python. Cursor AI is used to generate a program that creates a text file, writes sample content into it, reads the stored data, and displays the output. This task highlights how Cursor AI assists in quickly generating functional code and is useful for understanding file handling concepts in Python.

Task 3: CSV Data Analysis

- ❖ **Scenario:**
You are processing structured data from a CSV file.
- ❖ **Task:**
Use Gemini in Colab to read a CSV file and calculate mean, min, and max.
- ❖ **Expected Output:**
 - Correct output
 - Screenshot

Prompt

Write Python code to read a CSV file and calculate mean, minimum, and maximum values.

The screenshot displays a Jupyter Notebook interface with a dark theme. The left sidebar shows a file explorer with 'sample_data' and 'sample.txt'. The main area contains a Python script that reads a CSV file, calculates mean, min, and max for numerical columns, and prints the results. The output shows the first 5 rows of the dataset and calculated statistics.

```
# Import necessary libraries
import pandas as pd

# Define the path to your CSV file
csv_file_path = "sample_data.csv"

# Read the CSV file into a pandas DataFrame
df = pd.read_csv(csv_file_path)

# Display the first few rows of the dataset to understand its structure
print(df.head())

# Select only numerical columns for calculations
numerical_df = df.select_dtypes(include="number")

# Calculate mean, minimum, and maximum for numerical columns
mean_values = numerical_df.mean()
min_values = numerical_df.min()
max_values = numerical_df.max()

# Print the results
print("Calculated Statistics (Mean, Min, Max):")
print(mean_values)
print(min_values)
print(max_values)

# Print the numerical columns found in the CSV to calculate statistics.
```

Successfully loaded data from 'sample_data.csv'.

First 5 rows of the dataset:

Duration	Pulse	Resective	Calories
0	100	120	400.1
1	110	130	450.5
2	120	140	500.0
3	130	150	550.2
4	140	160	600.0

Calculated Statistics (Mean, Min, Max):

Duration	Pulse	Resective	Calories
Mean	125.0	145.0	525.0
Min	100.0	120.0	400.1
Max	140.0	160.0	600.0

```
***
Duration    63.846154
Pulse       107.461538
Maxpulse    134.047337
Calories    375.790244

dtype: float64

Minimum values:
0
Duration    15.0
Pulse       80.0
Maxpulse    100.0
Calories     50.3

dtype: float64

Maximum values:
0
Duration    300.0
Pulse      159.0
Maxpulse    184.0
Calories   1860.4

dtype: float64
```

Code Explanation:

1. `import pandas as pd` : Imports the pandas library, which is essential for working with DataFrames (table-like data structures).
2. `csv_file_path = '/content/data.csv'` : Defines the path to your CSV file. Make sure this path is correct.
3. `try...except` block: This block is used for robust error handling:
 - `df = pd.read_csv(csv_file_path)` : Attempts to read the CSV file into a pandas DataFrame named `df`.
 - `display(df.head())` : Shows the first 5 rows of the loaded DataFrame. This helps verify that the data was loaded correctly and gives an idea of its structure.
 - `numerical_df = df.select_dtypes(include=['number'])` : Filters the DataFrame to include only columns with numerical data types (integers and floats). Statistical calculations only make sense for numerical columns.
 - `mean_values = numerical_df.mean()` : Calculates the mean (average) for each numerical column.
 - `min_values = numerical_df.min()` : Calculates the minimum value for each numerical column.
 - `max_values = numerical_df.max()` : Calculates the maximum value for each numerical column.
 - `display(...)` : Prints the calculated mean, minimum, and maximum values for each respective numerical column.
 - **Error Handling:** Catches `FileNotFoundError` if the CSV doesn't exist, `EmptyDataError` if the file is empty, and a general `Exception` for any other issues, providing informative messages.

Explanation

- Pandas used for CSV handling
- `mean()`, `min()`, `max()` functions applied
- Displays statistical values

Description

In this task, Google Gemini is used in Google Colab to process structured data from a CSV file. The generated Python code reads the CSV file and calculates statistical values such as mean, minimum, and maximum. This task demonstrates how AI tools simplify data analysis tasks and help users apply Python libraries like Pandas for real-world data processing.

Task 4: Sorting Lists – Manual vs Built-in

❖ **Scenario:**

You are reviewing algorithm choices for efficiency.

❖ **Task:**

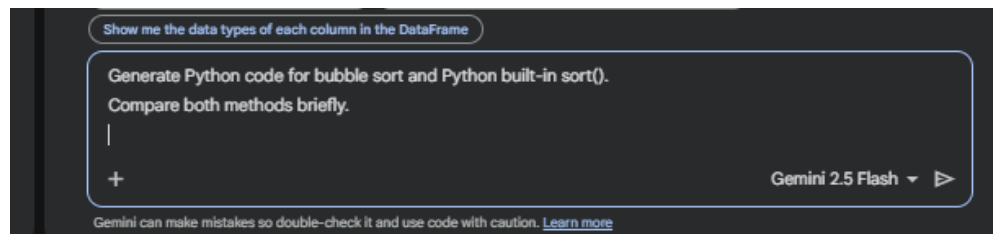
Use **Gemini** to generate:

- Bubble sort
- Python's built-in sort()
- Compare both implementations.

❖ **Expected Output:**

- Two versions of code
- Short comparison

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.



Prompt

Generate Python code for bubble sort and Python built-in sort().
Compare both methods briefly.

First, let's implement the **Bubble Sort** algorithm. It's a simple comparison-based sorting algorithm.

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Test Bubble Sort
my_list = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {my_list}")
sorted_list_bubble = bubble_sort(list(my_list)) # Use a copy to avoid modifying original
print(f"Sorted using Bubble Sort: {sorted_list_bubble}")
```

Next, let's use Python's **built-in sort()** method (for lists) or **sorted()** function (for any iterable). These are highly optimized and generally preferred in practice.

```
# Test Python's built-in sort()
my_list_2 = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {my_list_2}")
my_list_2.sort() # Sorts the list in-place
print(f"Sorted using built-in sort() method: {my_list_2}")

# Test Python's built-in sorted() function
my_tuple = (64, 34, 25, 12, 22, 11, 90)
print(f"Original tuple: {my_tuple}")
sorted_tuple = sorted(my_tuple) # Returns a new sorted list from an iterable
print(f"Sorted using built-in sorted() function: {sorted_tuple}")
```

Gemini

Comparison of Bubble Sort vs. Python's Built-in Sort

Feature	Bubble Sort	Python's Built-in sort() / sorted()
Algorithm	Simple comparison-based, repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.	Timsort (a hybrid stable sorting algorithm, derived from merge sort and insertion sort). Highly optimized.
Time Complexity	Worst/Average: $O(n^2)$ (very inefficient for large datasets). Best: $O(n)$ (if already sorted).	Worst/Average/Best: $O(n \log n)$ (very efficient for large datasets).
Space Complexity	$O(1)$ (in-place sort).	$O(n)$ in worst case (for Timsort, due to temporary storage).
Stability	Stable (maintains the relative order of equal elements).	Stable.
Ease of Use	Requires manual implementation.	Very easy to use; built directly into Python.
When to Use	Primarily for educational purposes to understand basic sorting concepts. Not recommended for production.	Almost always preferred for general-purpose sorting in Python due to its efficiency and reliability.

In summary: While Bubble Sort is easy to understand, Python's built-in `sort()` method (and `sorted()` function) are significantly more efficient, robust, and should be used for practical applications.

Sure! I'll generate the Python code for both bubble sort and the built-in `sort()` function, and then provide a brief comparison of these two sorting methods.

Great! Both the Bubble Sort implementation and the demonstrations of Python's built-in `sort()` method and `sorted()` function have executed successfully.

Bubble Sort:

- Original list: [64, 34, 25, 12, 22, 11, 90]
- Sorted using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]

Python's Built-in Sort:

- Original list (my_list_2): [64, 34, 25, 12, 22, 11, 90]
- Sorted using sort() method: [11, 12, 22, 25, 34, 64, 90]
- Original tuple (my_tuple): (64, 34, 25, 12, 22, 11, 90)
- Sorted using sorted() function: [11, 12, 22, 25, 34, 64, 90]

As you can see, both methods correctly sorted the provided lists/tuples. Now, let's briefly compare them:

Comparison of Bubble Sort vs. Python's Built-in Sort

Feature	Bubble Sort	Python's Built-in sort() / sorted()
Algorithm	Simple comparison-based, repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.	Timsort (a hybrid stable sorting algorithm, derived from merge sort and insertion sort). Highly optimized.
Time Complexity	Worst/Average: $O(n^2)$ (very inefficient for large datasets). Best: $O(n)$ (if already sorted).	Worst/Average/Best: $O(n \log n)$ (very efficient for large datasets).
Space Complexity	$O(1)$ (in-place sort).	$O(n)$ in worst case (for Timsort, due to temporary storage).
Stability	Stable (maintains the relative order of equal elements).	Stable.
Ease of Use	Requires manual implementation.	Very easy to use; built directly into Python.
When to Use	Primarily for educational purposes to understand basic sorting concepts. Not recommended for production.	Almost always preferred for general-purpose sorting in Python due to its efficiency and reliability.

In summary: While Bubble Sort is easy to understand, Python's built-in `sort()` method (and `sorted()` function) are significantly more efficient, robust, and should be used for practical applications.

We've successfully generated Python code for both the Bubble Sort algorithm and Python's highly optimized built-in `sort()` method and `sorted()` function. We also executed them to demonstrate their functionality, and provided a detailed comparison highlighting their differences in terms of algorithm, time/space complexity, stability, and ease of use.

Is there anything else I can assist you with regarding sorting, or perhaps a different programming task?

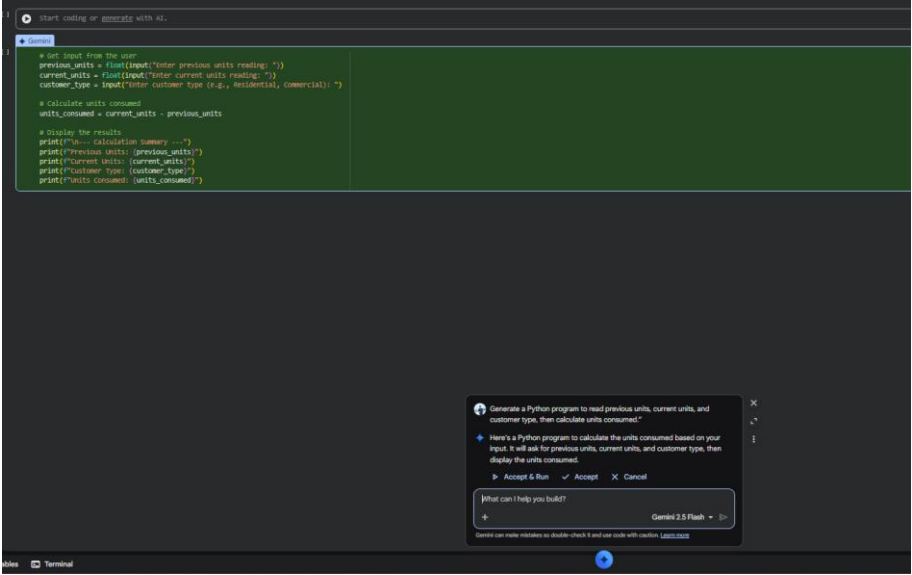
Final Comparison Section

Tool	Strengths	Weakness
Gemini	Good explanations, works well in Colab	Needs internet

	Cursor AI GitHub Copilot	Fast code generation & refactoring Inline suggestions	Editor based Limited explanation	
Description This task compares two sorting approaches generated by Google Gemini: a manual sorting algorithm (Bubble Sort) and Python’s built-in sort() method. The purpose of this task is to analyze the efficiency and readability of different sorting techniques. Gemini helps in generating both implementations and provides a brief comparison, showing why built-in methods are more efficient for practical applications.				

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/I	Regulation	R23
Date and Day of Assignment	Week 2 - Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.3(Present assignment number)/24(Total number of assignments)			

Question	Expected Time to complete
1 Lab 3: Application for TGNPDCL – Electricity Bill Generation Using Python & AI Tools Lab Objectives - To design a real-world electricity billing application using Python - To use AI-assisted coding tools for logic generation and optimization - To understand conditional logic and arithmetic operations	Week2 - Wednesday

• To generate structured billing output similar to utility bills Lab Outcomes (LOs) After completing this lab, students will be able to: • Read and validate user input in Python • Apply conditional logic for tariff-based billing • Use AI tools to assist in program development • Calculate and display electricity bill components • Build a complete real-time application	
Task 1: AI-Generated Logic for Reading Consumer Details Scenario An electricity billing system must collect accurate consumer data. Task Description Use an AI tool (GitHub Copilot / Gemini) to generate a Python program that: • Reads: ○ Previous Units (PU) ○ Current Units (CU) ○ Type of Customer • Calculates units consumed • Implements logic directly in the main program (no functions) Expected Output • Correct input reading • Units consumed calculation • Screenshot showing AI-generated code • Sample input and output Prompt Generate a Python program to read previous units, current units, and customer type, then calculate units consumed.”  The screenshot shows a code editor with a dark theme. The top bar has a toggle for 'Start coding or generate with AI.' and a 'Generate' button. The code editor contains the following Python code: <pre># Get input from the user previous_units = float(input("Enter previous units reading: ")) current_units = float(input("Enter current units reading: ")) customer_type = input("Enter customer type (e.g., residential, commercial): ") # Calculate units consumed units_consumed = current_units - previous_units # Display the results print("\n... calculation summary ...") print(f"Previous units: {previous_units}") print(f"Current units: {current_units}") print(f"Customer type: {customer_type}") print(f"Units consumed: {units_consumed}")</pre> Below the code editor is a chat window for Gemini 2.0 Flash. The prompt is: "Generate a Python program to read previous units, current units, and customer type, then calculate units consumed." The response is: "Here's a Python program to calculate the units consumed based on your input. It will ask for previous units, current units, and customer type, then display the units consumed." The chat window has buttons for 'Accept & Run', 'Accept', and 'Cancel'. At the bottom of the chat window is a text input field with the placeholder 'What can I help you build?' and a 'Gemini 2.0 Flash' label.	

```
Start coding or generate with AI.

◆ Gemini

# Get input from the user
previous_units = float(input("Enter previous units reading: "))
current_units = float(input("Enter current units reading: "))
customer_type = input("Enter customer type (e.g., Residential, Commercial): ")

# Calculate units consumed
units_consumed = current_units - previous_units

# Display the results
print(f"\n--- Calculation Summary ---")
print(f"Previous Units: {previous_units}")
print(f"Current Units: {current_units}")
print(f"Customer Type: {customer_type}")
print(f"Units Consumed: {units_consumed}")

Enter previous units reading: 4
Enter current units reading: 234
Enter customer type (e.g., Residential, Commercial): 123

--- Calculation Summary ---
Previous Units: 4.0
Current Units: 234.0
Customer Type: 123
Units Consumed: 230.0
```

Description

This task collects electricity consumer details such as previous meter reading, current meter reading, and customer type.
The program calculates the number of units consumed by subtracting previous units from current units.
All logic is written directly in the main program.

Task 2: Energy Charges Calculation Based on Units Consumed

Scenario

Energy charges depend on the number of units consumed and customer type.

Task Description

Review the AI-generated code from Task 1 and extend it to:

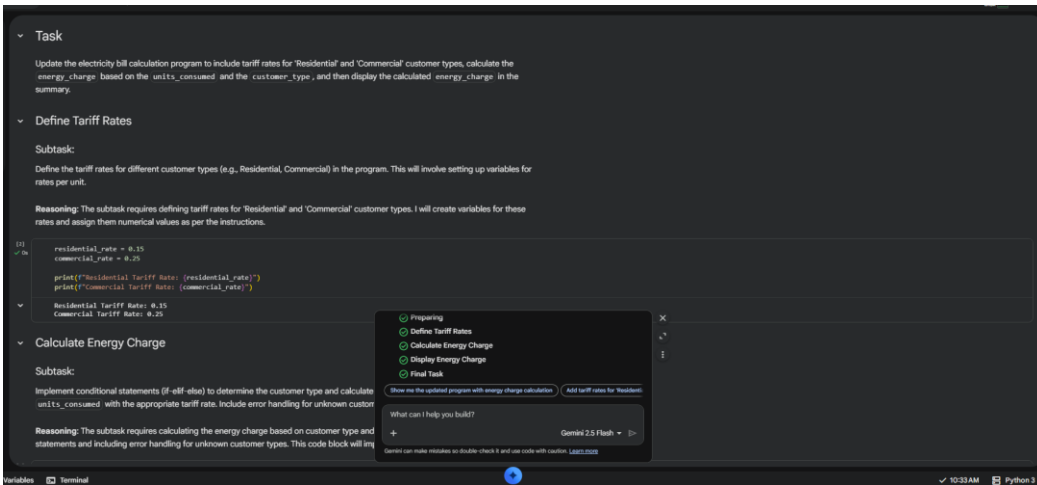
- Calculate **Energy Charges (EC)**
- Use conditional statements based on:
 - Domestic
 - Commercial
 - Industrial consumers
- Improve readability using AI prompts such as:
 - “Simplify energy charge calculation logic”
 - “Optimize conditional statements”

Expected Output

- Correct EC calculation
- Clear conditional logic
- Original and improved versions (optional)
- Sample execution results

AI Prompt

“Extend the program to calculate energy charges using conditional statements based on customer type and optimize the logic.”



Description

Energy charges are calculated based on the number of units consumed and the type of customer.

Different tariffs are applied for Domestic, Commercial, and Industrial consumers using conditional statements.

Task 3: Modular Design Using AI Assistance (Using Functions)

Scenario

Billing logic must be reusable for multiple consumers.

Task Description

Use AI assistance to generate a Python program that:

- Uses user-defined functions to:
 - Calculate Energy Charges

- Calculate Fixed Charges

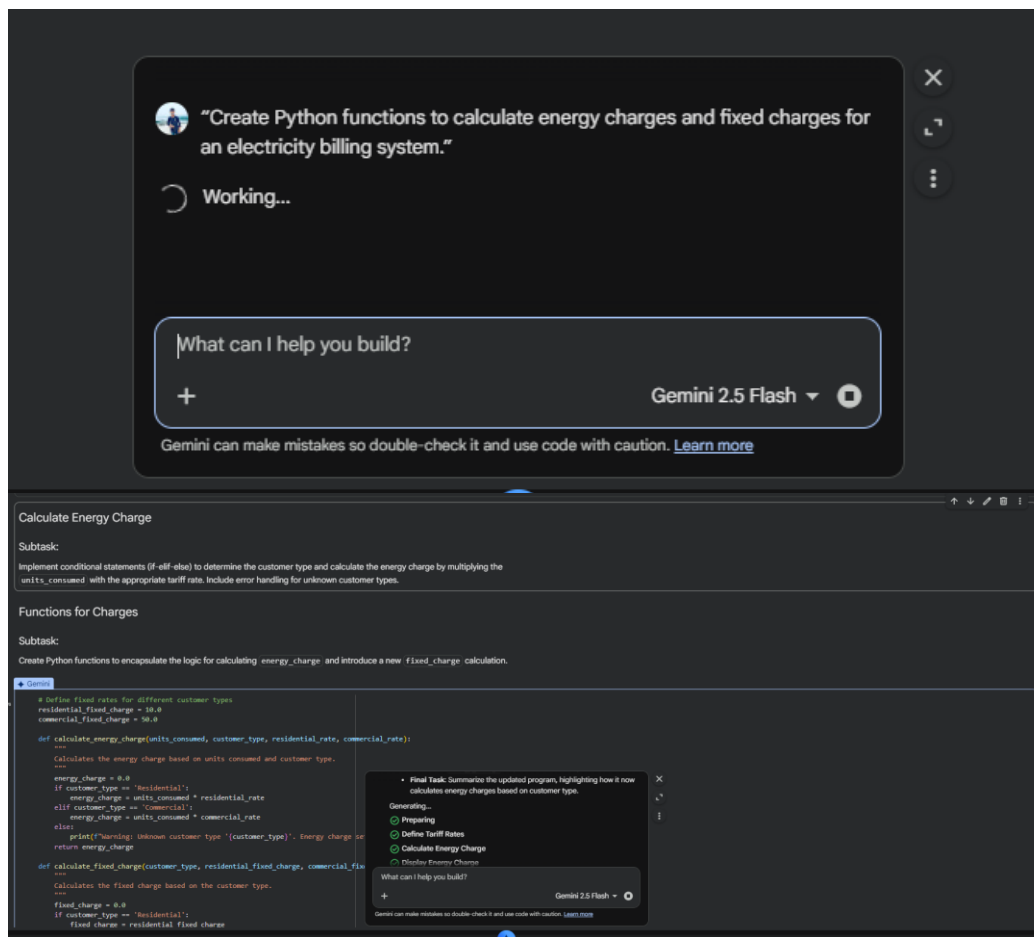
- Returns calculated values
- Includes meaningful comments

Expected Output

- Function-based Python program
- Correct EC and FC values
- Screenshots of AI-assisted function generation
- Test cases with outputs

AI Prompt

“Create Python functions to calculate energy charges and fixed charges for an electricity billing system.”



+ Code + Text Final Task Subtask: Summarize the updated program, highlighting how it now calculates energy charges based on customer type. Summary: Q&A The updated program now calculates energy charges by first defining specific tariff rates for 'Residential' (\$0.15 per unit) and 'Commercial' (\$0.25 per unit) customer types. It then uses conditional logic to apply the appropriate tariff rate to the <code>units_consumed</code> based on the <code>customer_type</code>. If an unknown customer type is encountered, the energy charge is set to \$0.0, and an error message is displayed. Finally, the calculated <code>energy_charge</code> is included in the updated calculation summary. Data Analysis Key Findings • Tariff rates were successfully defined: \$0.15 per unit for 'Residential' customers and \$0.25 per unit for 'Commercial' customers. • The <code>energy_charge</code> calculation now dynamically applies the correct tariff rate based on the <code>customer_type</code> and <code>units_consumed</code> using <code>if-elif-else</code> statements. • The program includes error handling for unknown customer types, setting the <code>energy_charge</code> to \$0.0 and printing a notification message (e.g., "Unknown customer type: 123. Energy charge set to 0.0"). • The final summary output has been updated to explicitly display the calculated <code>energy_charge</code>. Insights or Next Steps • The program now accurately reflects different pricing structures for various customer segments, enhancing billing fairness and transparency. • Consider expanding the program to include more customer types, tiered pricing structures, or additional charges (e.g., fixed charges, taxes) to create a more comprehensive billing system.	
To improve reusability and readability, the billing logic is divided into user-defined functions. Separate functions are created for calculating energy charges and fixed charges. Description Additional charges such as Fixed Charges (FC), Customer Charges (CC), and Electricity Duty (ED) are calculated. Electricity Duty is calculated as 5% of Energy Charges.	
Task 4: Calculation of Additional Charges Scenario Electricity bills include multiple additional charges. Task Description Extend the program to calculate: • FC – Fixed Charges • CC – Customer Charges • ED – Electricity Duty (percentage of EC) Use AI prompts like: • <i>"Add electricity duty calculation"</i> • <i>"Improve billing accuracy"</i> Expected Output • Individual charge values printed	

- Correct duty calculation
- Well-structured output
- Verified intermediate results

AI Prompt

"Generate a final electricity bill showing all charges in a formatted output."

Description

The final bill amount is calculated by summing Energy Charges, Fixed Charges, Customer Charges, and Electricity Duty.

The output is displayed in a structured format similar to a real electricity bill.

```

Generate a final electricity bill showing all charges in a formatted output.

def generate_bill(customer_type, units_consumed):
    print("\n--- Final Electricity Bill ---")
    print(f"Customer Type: {customer_type}")
    print(f"Units Consumed: {units_consumed}")
    print(f"Energy Charge: ${calculate_energy_charge(units_consumed)}")
    print(f"Fixed Charge: ${calculate_fixed_charge(customer_type)}")
    print(f"Total Bill: ${total_bill(units_consumed, customer_type)}")

# Example usage
customer_type = "residential"
units_consumed = 250
generate_bill(customer_type, units_consumed)

--- Final Electricity Bill ---
Customer Type: residential
Units Consumed: 250
Energy Charge: $8.00
Fixed Charge: $8.00
Total Bill: $16.00

Functions for Charges

Subtask:
Create Python functions to encapsulate the logic for calculating energy_charge and introduce a new fixed_charge calculation.

def calculate_energy_charge(units_consumed, customer_type, residential_rate, commercial_rate):
    """
    Calculates the energy charge based on units consumed and customer type.
    """
    if customer_type == "residential":
        energy_charge = units_consumed * residential_rate
    else:
        energy_charge = units_consumed * commercial_rate
    return energy_charge

def calculate_fixed_charge(customer_type):
    """
    Calculates the fixed charge based on customer type.
    """
    if customer_type == "residential":
        fixed_charge = 8.0
    else:
        fixed_charge = 16.0
    return fixed_charge

def total_bill(units_consumed, customer_type):
    """
    Calculates the total bill by summing energy charge and fixed charge.
    """
    energy_charge = calculate_energy_charge(units_consumed, customer_type, residential_rate, commercial_rate)
    fixed_charge = calculate_fixed_charge(customer_type)
    total_bill = energy_charge + fixed_charge
    return total_bill
  
```

Task 5: Final Bill Generation and Output Analysis Scenario

The final electricity bill must present all values clearly.

Task Description

Develop the final Python application to:

- Calculate total bill:
- Total Bill = EC + FC + CC + ED
- Display:
 - Energy Charges (EC)
 - Fixed Charges (FC)
 - Customer Charges (CC)
 - Electricity Duty (ED)
 - Total Bill Amount
- Analyze the program based on:
 - Accuracy
 - Readability
 - Real-world applicability

Expected Output

- Complete electricity bill output
- Neatly formatted display
- Sample input/output

- Short analysis paragraph

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments, & code explanation, and output and if required, screenshots.

Program Analysis

- **Accuracy:**
The program correctly calculates the total electricity bill using all required components and follows the specified formula.
- **Readability:**
Clear variable names and structured output make the program easy to understand and maintain.
- **Real-World Applicability:**
The bill format and calculations closely resemble real electricity billing systems such as TGNPDCL.

Final Task

Subtask:

Summarize the updated program, highlighting how it now calculates a comprehensive electricity bill including energy charges, fixed charges, customer charges, and electricity duty.

Summary:

Q&A

The updated program now calculates a comprehensive electricity bill by first defining `customer_charge` and `electricity_duty` variables for different customer types (Residential and Commercial) with specific values. It then incorporates these into the billing process through dedicated functions, `calculate_customer_charge` and `calculate_electricity_duty`. These functions determine the applicable charge based on the customer type, similar to how energy and fixed charges are handled. Finally, the program calculates the `total_bill` by summing `energy_charge`, `fixed_charge`, `customer_charge`, and `electricity_duty`, and presents all these individual components along with the comprehensive total in a formatted output.

Data Analysis Key Findings

- Specific charges for customer type were defined: `residential_customer_charge` at \$5.0, `commercial_customer_charge` at \$15.0, `residential_electricity_duty` at \$2.0, and `commercial_electricity_duty` at \$10.0.
- Two new functions, `calculate_customer_charge` and `calculate_electricity_duty`, were successfully implemented to determine these charges based on customer type. These functions also include handling for unknown customer types, returning \$0.00 and printing a warning.
- During a test with an unknown customer type ("123"), both `calculate_customer_charge` and `calculate_electricity_duty` correctly returned \$0.00 and displayed warning messages.
- The `total_bill` calculation was updated to sum `calculated_energy_charge`, `calculated_fixed_charge`, `calculated_customer_charge`, and `calculated_electricity_duty`. In the provided execution context, the `total_bill` was \$0.00, implying the individual calculated charges were zero.
- The program now displays a formatted final electricity bill, detailing `Customer Type`, `Units Consumed`, `Energy Charge`, `Fixed Charge`, `Customer Charge`, `Electricity Duty`, and the `Total Bill`, all presented with two decimal places.

Insights or Next Steps

- The program successfully modularized the calculation of different charges and integrated them into a comprehensive bill.

Define Additional

Update Calculati

Calculate Total B

Display Final Elec

Final Task

Explain the Python code that

What can I help you build

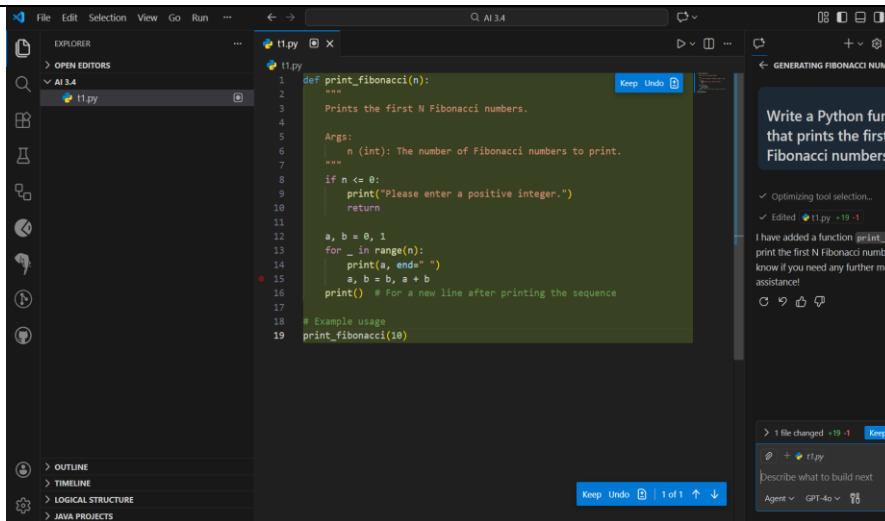
+

Gemini can make mistakes so do

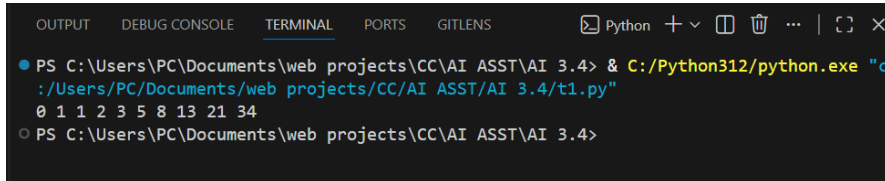
--	--	--

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025- 2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002 PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week2	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 3.4 (Present assignment number)/24(Total number of assignments)																				
	Question		Expected Time to co																	

		<i>complete</i>
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques Task 1: Zero-shot Prompt – Fibonacci Series Generator Task Description #1 • Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers. Expected Output #1 • A complete Python function generated by Copilot without any example provided. • Correct output for sample input N = 7 → 0 1 1 2 3 5 8 • Observation on how Copilot understood the instruction with zero context. Prompt : Write a Python function that prints the first N Fibonacci numbers.	Week 2



```
1 def print_fibonacci(n):
2     """
3     Prints the first N Fibonacci numbers.
4
5     Args:
6         n (int): The number of Fibonacci numbers to print.
7
8     """
9     if n <= 0:
10        print("Please enter a positive integer.")
11        return
12
13    a, b = 0, 1
14    for _ in range(n):
15        print(a, end=" ")
16        a, b = b, a + b
17        print() # For a new line after printing the sequence
18
19 # Example usage
20 print_fibonacci(10)
```



```
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4> & C:/Python312/python.exe "c:/Users/PC/Documents/web projects/CC/AI ASST/AI 3.4/t1.py"
0 1 1 2 3 5 8 13 21 34
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4>
```

Observation

In zero-shot prompting, no example was provided.
GitHub Copilot correctly understood:

- Fibonacci logic
- Loop structure
- Output formatting

This shows Copilot can generate functional code even with **minimal instructions**.

Task 2: One-shot

Prompt – List Reversal Function

Task Description #2

- Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.

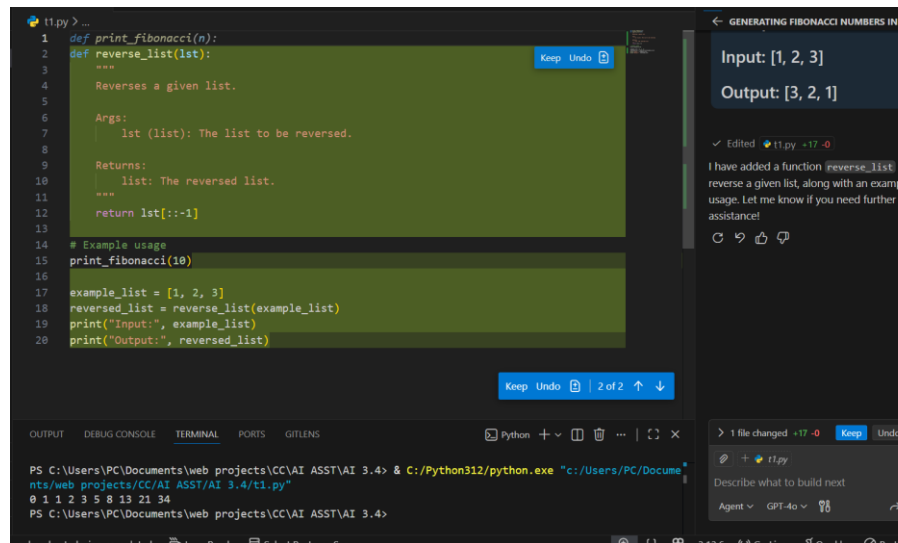
Prompt

Write a Python function to reverse a list.

Example:

Input: [1, 2, 3]

Output: [3, 2, 1]



```
t1.py > ...
1 def print_fibonacci(n):
2 def reverse_list(list):
3     """
4     Reverses a given list.
5
6     Args:
7         list (list): The list to be reversed.
8
9     Returns:
10        list: The reversed list.
11    """
12    return list[::-1]
13
14 # Example usage
15 print_fibonacci(10)
16
17 example_list = [1, 2, 3]
18 reversed_list = reverse_list(example_list)
19 print("Input:", example_list)
20 print("Output:", reversed_list)
```

GENERATING FIBONACCI NUMBERS IN ...

Input: [1, 2, 3]
Output: [3, 2, 1]

✓ Edited t1.py +17 -0

I have added a function `reverse_list` to reverse a given list, along with an example usage. Let me know if you need further assistance!

1 file changed +17 -0

Describe what to build next

Agent GPT-4o

PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4> & C:/Python312/python.exe "c:/Users/PC/Docume
nts/web projects/CC/AI ASST/AI 3.4/t1.py"
0 1 1 2 3 5 8 13 21 34
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4>

```
PS C:\Users\PC\Documents\web_projects\CC\AI ASST\AI 3.4> & C:/Python
exe "c:/Users/PC/Documents/web_projects/CC/AI ASST/AI 3.4/t1.py"
0 1 1 2 3 5 8 13 21 34
PS C:\Users\PC\Documents\web_projects\CC\AI ASST\AI 3.4> |
```

Observation

Adding **one example** helped Copilot:

- Choose slicing instead of complex logic
- Produce shorter and cleaner code

One-shot prompting improves **accuracy and simplicity**.

Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

- A function is_valid() that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.

Prompt

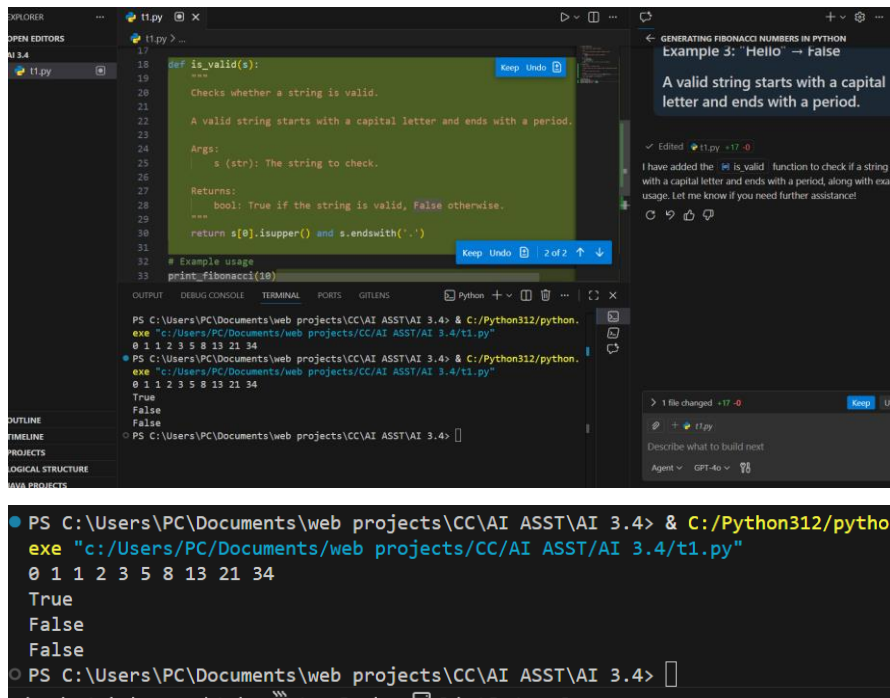
Write a function `is_valid(s)` that checks whether:

Example 1: "Hello." → True

Example 2: "hello." → False

Example 3: "Hello" → False

A valid string starts with a capital letter and ends with a period.



The screenshot shows a VS Code editor with a file named `t1.py`. The code defines a function `is_valid(s)` that checks if a string starts with a capital letter and ends with a period. The function returns `True` if both conditions are met, otherwise it returns `False`. The code also includes an example usage: `print(is_valid('Hello'))`. The terminal output shows the execution of the script, which prints `True` for the example usage.

```
def is_valid(s):  
    """  
    Checks whether a string is valid.  
  
    A valid string starts with a capital letter and ends with a period.  
  
    Args:  
        s (str): The string to check.  
  
    Returns:  
        bool: True if the string is valid, False otherwise.  
    """  
    return s[0].isupper() and s.endswith('.')  
  
# Example usage  
print(is_valid('Hello'))
```

```
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4> & C:/Python312/python.  
exe "c:/Users/PC/Documents/web projects/CC/AI ASST/AI 3.4/t1.py"  
0 1 1 2 3 5 8 13 21 34  
True  
False  
False  
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4>
```

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

Expected Output #4

- Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

- Submit both code versions and note how few-shot improves reliability.

Zero-shot Prompt

Write a Python function to validate an email address.

```
34 def is_valid_email(email):
35     email_regex = r'[a-zA-Z0-9._-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\s$'
36     return re.match(email_regex, email) is not None
37
38 # Example usage
39 print(is_valid_email("hello@domain.com")) # True
40 print(is_valid_email("hello")) # False
41 print(is_valid_email("Hello")) # False
42 print(is_valid_email("example@example.com")) # True
43 print(is_valid_email("invalid-email")) # False
```

Comparison Observation

Aspect	Zero-shot	Few-shot
Logic	Very basic	Detailed & reliable
Accuracy	Low	High
Validation	Weak	Strong

Few-shot prompting clearly **improves reliability**.

Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

- Experiment with 2 different prompt styles to generate a function that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

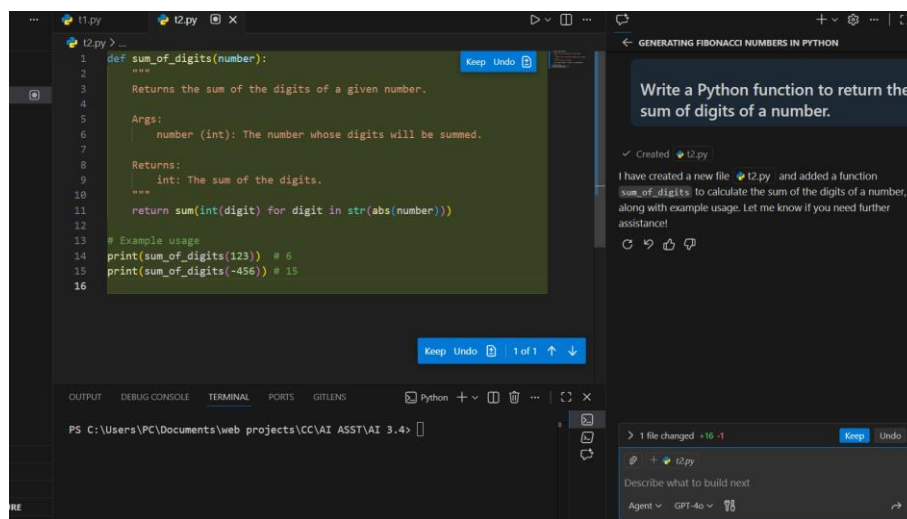
Expected Output #5

- Two versions of the `sum_of_digits()` function.
- Example Output: `sum_of_digits(123) → 6`
- Short analysis: which prompt produced cleaner or more optimized code and why?

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

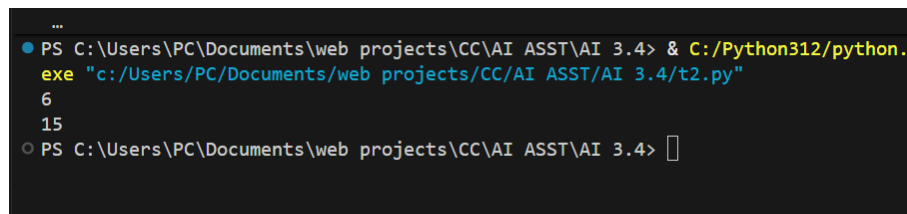
Style 1: Generic Prompt

Write a Python function to return the sum of digits of a number.



The screenshot shows a code editor with a Python file named `t2.py`. The code defines a function `sum_of_digits` that takes a number and returns the sum of its digits. The function uses `str(abs(number))` to convert the number to a string and iterate over its characters. The AI assistant's response is displayed on the right, showing the prompt and the generated code. The terminal at the bottom shows the execution of the code, which outputs `6` for `sum_of_digits(123)` and `15` for `sum_of_digits(-456)`.

```
1 def sum_of_digits(number):
2     """
3     Returns the sum of the digits of a given number.
4
5     Args:
6         number (int): The number whose digits will be summed.
7
8     Returns:
9         int: The sum of the digits.
10    """
11    return sum(int(digit) for digit in str(abs(number)))
12
13 # Example usage
14 print(sum_of_digits(123)) # 6
15 print(sum_of_digits(-456)) # 15
16
```



The screenshot shows a terminal window with the following commands and output:

```
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4> & C:/Python312/python.exe "c:/Users/PC/Documents/web projects/CC/AI ASST/AI 3.4/t2.py"
6
15
PS C:\Users\PC\Documents\web projects\CC\AI ASST\AI 3.4>
```

Analysis

Style-2 prompt produced:

- Cleaner code
- One-line optimized solution
- Better readability

👉 Prompt tuning improves code quality.

Final Conclusion

- **Zero-shot** → Works but may be basic
- **One-shot** → Improves clarity
- **Few-shot** → Best accuracy and logic
- **Prompt tuning** → Produces optimized code

GitHub Copilot responds better when **examples are included**.